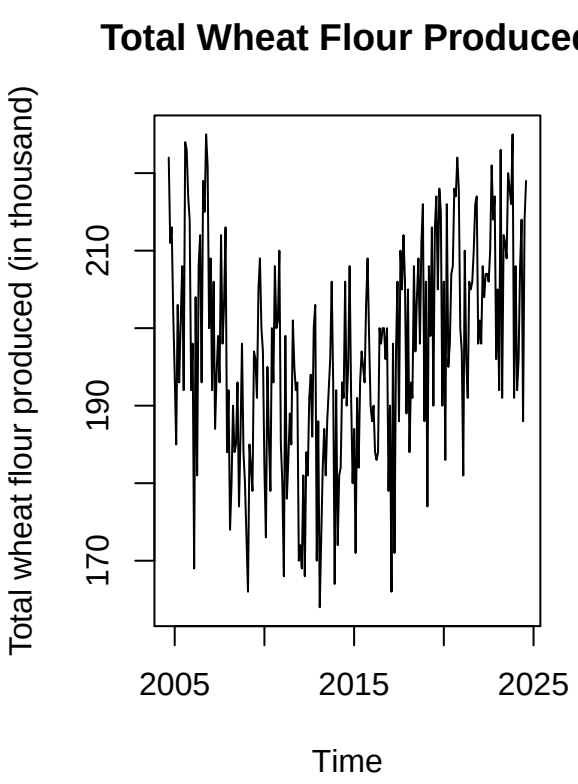
Codes_Group5

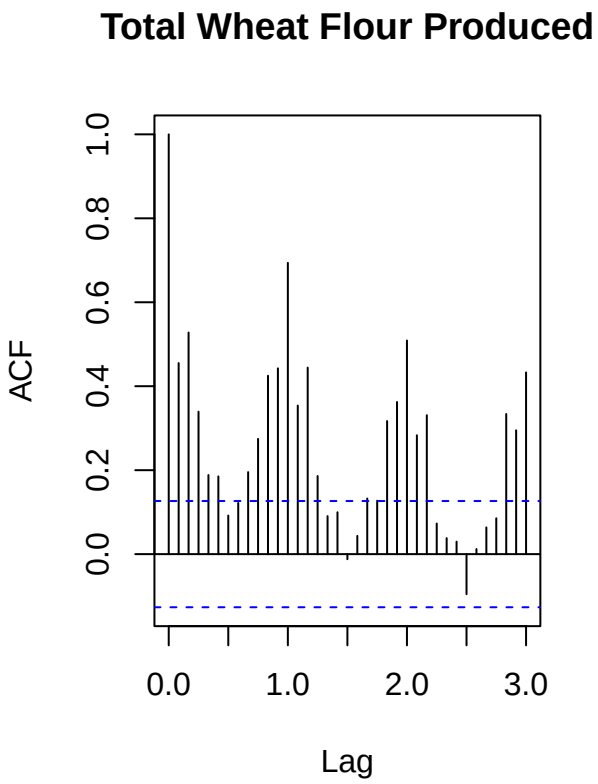
Chenxu Liu, Yuqi Li, Ronnie Min, Yulin Zhang

2024-11-27

Import Data

```
# Read the data
production <- read.csv("Data_Group 5.csv", header = TRUE)
wheatflour <- subset(production,
                     Milled.wheat.and.wheat.flour.produced == "Total wheat flour produced")
wheatflour = data.frame(wheatflour, row.names = NULL)
# Plot the time series data
wheatflourTS <- ts(wheatflour$VALUE, start = 2004 + 8/12, end = 2024 + 7/12, frequency = 12)
Time = time(wheatflourTS)
Season = as.factor(cycle(wheatflourTS))
par(mfrow = c(1, 2))
plot(wheatflourTS, ylab = "Total wheat flour produced (in thousand)",
     main = "Total Wheat Flour Produced")
acf(wheatflourTS, main = "Total Wheat Flour Produced", lag.max = 36)
```

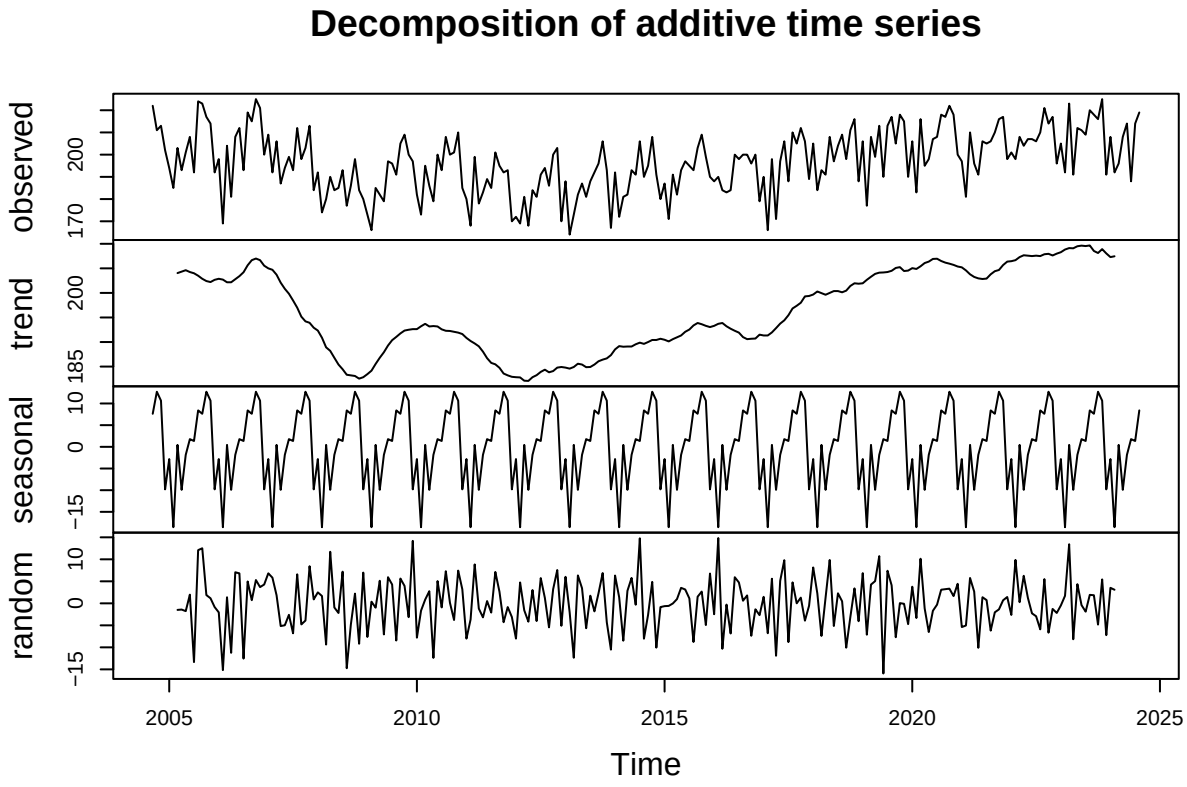





```
# Check if we need transformation on data
year = 20
group = rep(1: year, each = 12)
fligner.test(wheatflourTS, group)
```

```
##
##  Fligner-Killeen test of homogeneity of variances
##
## data:  wheatflourTS and group
## Fligner-Killeen:med chi-squared = 13.735, df = 19, p-value = 0.7989
```

```
# Then we decompose the data to see if the time series is stationary
plot(decompose(wheatflourTS))
```



Make training and validation sets

```
# Form training and validation sets
wheatflourTS.train <- window(wheatflourTS, start = 2004 + 8/12, end = 2023 + 7/12)
train.indx = 1:which(wheatflour$REF_DATE == "2023-08")
wheatflourTS.test <- window(wheatflourTS, start = 2023 + 8/12)
```

Regression Model (Un-regularized)

```
# First we check the un-regularized regression model
# Initialize all variables we need
APSE.regression = APSE.regression.both = c()
degree = 10
X = poly(Time, degree) # Polynomial matrix X
season <- as.factor(cycle(wheatflourTS.train))
season.test <- as.factor(cycle(wheatflourTS.test))
# For each polynomial degree p:
# (1) Fit a regression model based on training set
# (2) Predict the test set data based on the model
# (3) Calculate the APSE of predicting the test set
for (p in 1:degree) {
  # regression model with only trend
```

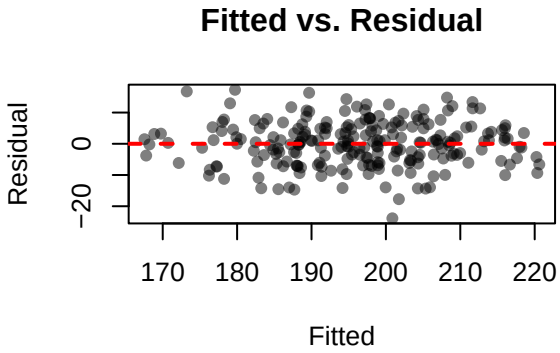
```
  fit.regression = lm(wheatflourTS.train ~ X[train.indx, 1:p])
  Fitted.test.regression = cbind(1, X[-train.indx, 1:p]) %*% coef(fit.regression)
  APSE.regression[p] = mean((as.vector(wheatflourTS.test) - Fitted.test.regression)^2)

  # regression model with trend + seasonality
  fit.regression.both = lm(wheatflourTS.train ~ X[train.indx, 1:p] + season)
  modelmatrix = model.matrix(wheatflourTS.test ~ X[-train.indx, 1:p] + season.test)
  Fitted.test.regression.both = coef(fit.regression.both) %*% t(modelmatrix)
  APSE.regression.both[p] = mean((as.vector(wheatflourTS.test) - Fitted.test.regression.both)^2)
}
data.frame("Degree" = 1:degree,
           "APSE_trend" = APSE.regression,
           "APSE_trend_and_seasonality)" = APSE.regression.both)
```

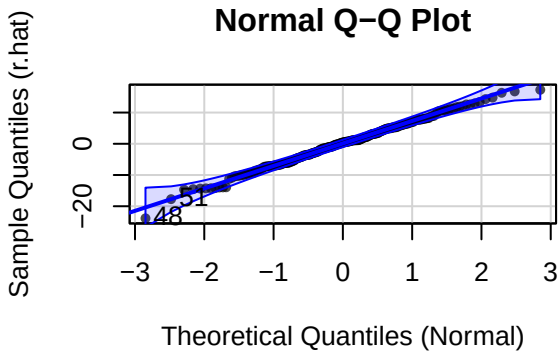
```
##    Degree APSE_trend APSE_trend_and_seasonality.
## 1       1   176.3305                   110.41617
## 2       2   287.1920                   221.17603
## 3       3   155.0472                    93.66891
## 4       4   147.0197                    84.47821
## 5       5   150.9064                    92.86942
## 6       6   160.5185                    89.38806
## 7       7   382.9522                   366.50560
## 8       8   544.4073                   265.92936
## 9       9   169.8882                    92.43341
## 10     10  4431.2062                  1475.96365
```

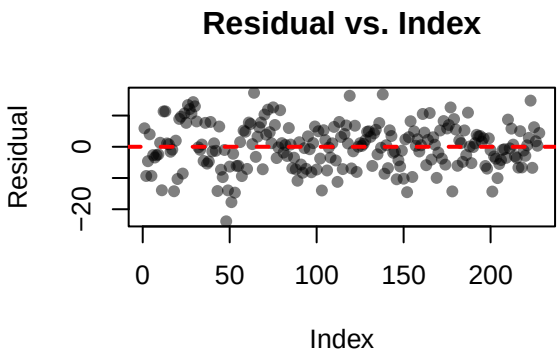
```
# We choose the trend + seasonality regression model with p = 4
# We fit the optimal model to the training data
p.optimal = which.min(APSE.regression)
fit.optimal = lm(wheatflourTS.train ~ X[train.indx, 1:p.optimal] + season)

# Perform model diagnostics to check the fit performance
par(mfrow = c(2, 2))
residuals = fit.optimal$residuals
fitted = fit.optimal$fitted.values
plot(fitted, residuals, pch = 16, col = adjustcolor("black", 0.5),
     xlab = "Fitted", ylab = "Residual", main = "Fitted vs. Residual")
abline(h = 0, lty = 2, lwd = 2, col = "red")
car::qqPlot(residuals, pch = 16, col = adjustcolor("black", 0.7),
            xlab = "Theoretical Quantiles (Normal)", ylab = "Sample Quantiles (r.hat)",
            main = "Normal Q-Q Plot")
```

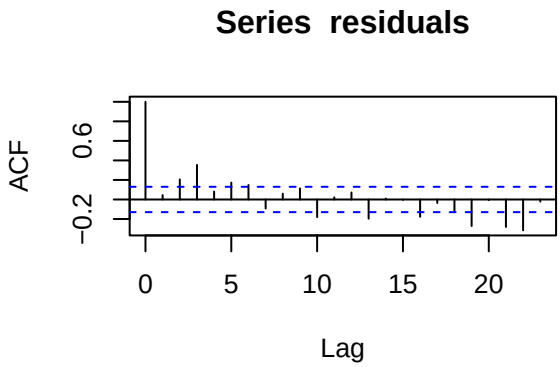
```
## [1] 48 51
```

```
plot(residuals, pch = 16, col = adjustcolor("black", 0.5),
     ylab = "Residual", main = "Residual vs. Index")
abline(h = 0, lty = 2, lwd = 2, col = "red")
acf(residuals)
```

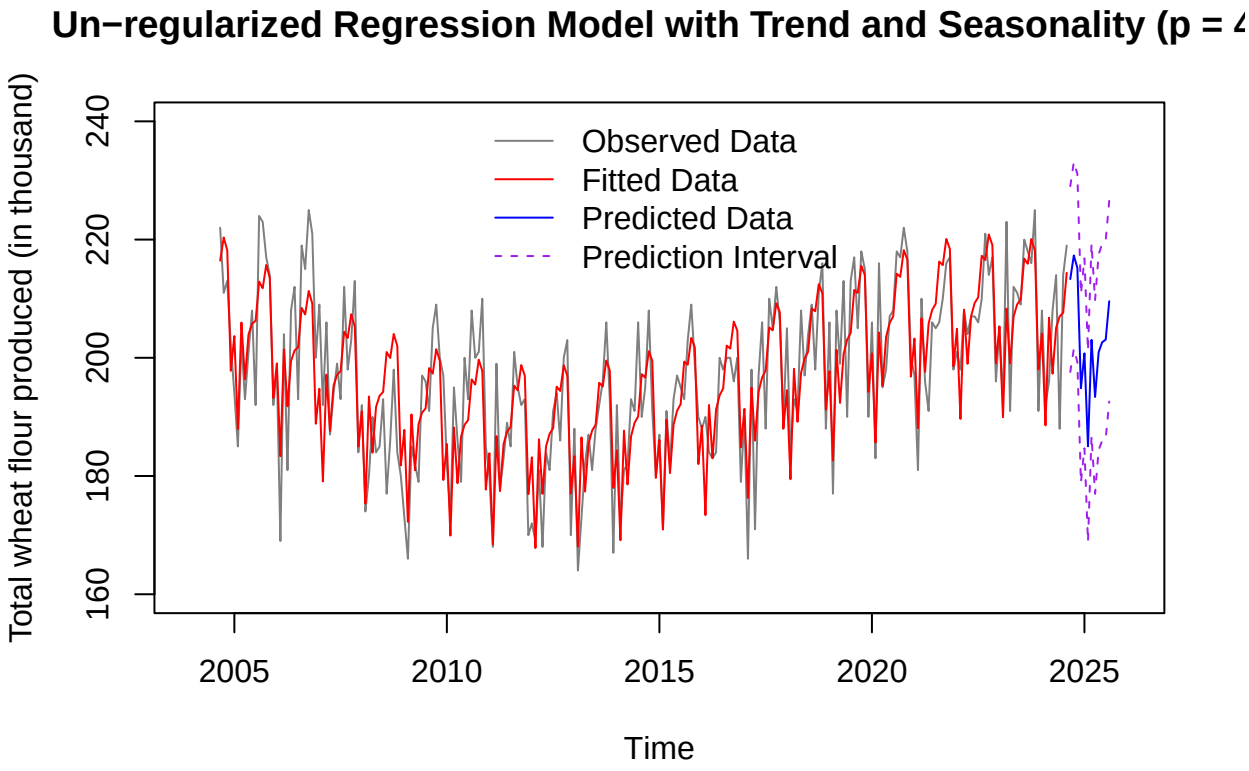









```r
# Plot the observed data, fitted data, predicted data and 95% prediction interval
time = as.vector(Time)
model.regression = lm(wheatflourTS ~ poly(Time, p.optimal) + Season) # final model
predict.time = seq(2024 + 8/12, 2025 + 7/12, length = 12)
predict = list(Time = predict.time, Season = as.factor(c(9:12, 1:8)))
Fitted.predict.regression = predict(model.regression, newdata = predict, interval = "prediction")
plot(wheatflourTS, xlim = c(2004, 2026), ylim = c(160, 240), col = adjustcolor("black", 0.5),
     ylab = "Total wheat flour produced (in thousand)",
     main = "Un-regularized Regression Model with Trend and Seasonality (p = 4)") # observed data
lines(time, model.regression$fitted.values, col = "red") # fitted data
lines(Fitted.predict.regression[, 1] ~ predict$Time, col = "blue") # predicted data
lines(Fitted.predict.regression[, 2] ~ predict$Time, lty = 2, col = "purple") # 95% prediction interval
lines(Fitted.predict.regression[, 3] ~ predict$Time, lty = 2, col = "purple") # 95% prediction interval
legend("top", legend = c("Observed Data", "Fitted Data", "Predicted Data", "Prediction Interval"),
       col = c(adjustcolor("black", 0.5), "red", "blue", "purple"), lty = c(1, 1, 1, 2), bty = "n")
```



Regression Model (Regularized)

```
# Then we check the regularized regression model
# Consider alpha = {0, 0.5, 1}
# Initialize all variables we need
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 5.0
```

```
APSE.ridge = APSE.elastic = APSE.lasso= c()
APSE.ridge.both = APSE.elastic.both = c()
lambda.ridge = lambda.elastic = lambda.lasso = c()
lambda.ridge.both = lambda.elastic.both = c()
poly.degree = 2:degree

# For each polynomial degree p:
# (1) Find the regression model with optimal lambda (lambda.1se) using cross validation method
# (2) Use the optimal lambda to fit the regression model based on training set
# (3) Predict the test set data based on the model
# (4) calculate the APSE of predicting the test set
# For Lasso regression, we fit the trend first and see if we needed to add seasonality model later
```

```r
for (p in 1:length(poly.degree)) {
  # seasonality matrix with only 0 and 1 for each season
  Seasonmatrix = model.matrix(wheatflourTS ~ Season)[, 2:12]
  set.seed(443) # set seed to ensure reproducible random variables

  # ridge regression model with only trend
  CV.ridge = cv.glmnet(X[train.indx, 1:poly.degree[p]], as.vector(wheatflourTS.train),
                       alpha = 0, nfolds = 10)
  lambda.ridge[p] = CV.ridge$lambda.1se
  fit.train.ridge = glmnet(X[train.indx, 1:poly.degree[p]], as.vector(wheatflourTS.train),
                           alpha = 0,
                           lambda = lambda.ridge[p],
                           standardize = TRUE, intercept = TRUE,
                           family = "gaussian")
  Fitted.test.ridge = predict(fit.train.ridge, newx = X[-train.indx, 1:poly.degree[p]],
                              type = "response")
  APSE.ridge[p] = mean((wheatflourTS.test - Fitted.test.ridge)^2)

  # ridge regression model with trend + seasonality
  CV.ridge.both = cv.glmnet(cbind(X[train.indx, 1:poly.degree[p]], Seasonmatrix[train.indx,]),
                            as.vector(wheatflourTS.train), alpha = 0, nfolds = 10)
  lambda.ridge.both[p] = CV.ridge.both$lambda.1se
  fit.train.ridge.both = glmnet(cbind(X[train.indx, 1:poly.degree[p]], Seasonmatrix[train.indx,]),
                                as.vector(wheatflourTS.train), alpha = 0,
                                lambda = lambda.ridge.both[p],
                                standardize = TRUE, intercept = TRUE,
                                family = "gaussian")
  Fitted.test.ridge.both = predict(fit.train.ridge.both, newx = cbind(X[-train.indx, 1:poly.degree[p]],
                                                                      Seasonmatrix[-train.indx,]),
                                   type = "response")
  APSE.ridge.both[p] = mean((wheatflourTS.test - Fitted.test.ridge.both)^2)

  # elastic net regression model with only trend
  CV.elastic = cv.glmnet(X[train.indx, 1:poly.degree[p]], as.vector(wheatflourTS.train),
                         alpha = 0.5, nfolds = 10)
  lambda.elastic[p] = CV.elastic$lambda.1se
  fit.train.elastic = glmnet(X[train.indx, 1:poly.degree[p]], as.vector(wheatflourTS.train),
                             alpha = 0.5,
                             lambda = lambda.elastic[p],
                             standardize = TRUE, intercept = TRUE,
                             family = "gaussian")
  Fitted.test.elastic = predict(fit.train.elastic, newx = X[-train.indx, 1:poly.degree[p]],
                                type = "response")
  APSE.elastic[p] = mean((wheatflourTS.test - Fitted.test.elastic)^2)

  # elastic net regression model with trend + seasonality
  CV.elastic.both = cv.glmnet(cbind(X[train.indx, 1:poly.degree[p]], Seasonmatrix[train.indx,]),
                              as.vector(wheatflourTS.train), alpha = 0.5, nfolds = 10)
  lambda.elastic.both[p] = CV.elastic.both$lambda.1se
  fit.train.elastic.both = glmnet(cbind(X[train.indx, 1:poly.degree[p]], Seasonmatrix[train.indx,]),
                                  as.vector(wheatflourTS.train), alpha = 0.5,
                                  lambda = lambda.elastic.both[p],
                                  standardize = TRUE, intercept = TRUE,
```

```
                                    family = "gaussian")
  Fitted.test.elastic.both = predict(fit.train.elastic.both,
                                     newx = cbind(X[-train.indx, 1:poly.degree[p]],
                                                  Seasonmatrix[-train.indx,]),
                                     type = "response")
  APSE.elastic.both[p] = mean((wheatflourTS.test - Fitted.test.elastic.both)^2)

  # For Lasso regression, we only consider trend case
  # lasso regression model with only trend
  CV.lasso = cv.glmnet(X[train.indx, 1:poly.degree[p]], as.vector(wheatflourTS.train),
                       alpha = 1, nfolds = 10)
  lambda.lasso[p] = CV.lasso$lambda.1se
  fit.train.lasso = glmnet(X[train.indx, 1:poly.degree[p]], as.vector(wheatflourTS.train),
                           alpha = 1,
                           lambda = lambda.lasso[p],
                           standardize = TRUE, intercept = TRUE,
                           family = "gaussian")
  Fitted.test.lasso = predict(fit.train.lasso, newx = X[-train.indx, 1:poly.degree[p]],
                              type = "response")
  APSE.lasso[p] = mean((wheatflourTS.test - Fitted.test.lasso)^2)
}

data.frame("Degree" = poly.degree,
           "APSE_Ridge_trend" = APSE.ridge,
           "APSE_Ridge_trend_and_seasonality" = APSE.ridge.both,
           "APSE_Elastic_trend" = APSE.elastic,
           "APSE_Elastic_trend_and_seasonality" = APSE.elastic.both,
           "APSE_Lasso" = APSE.lasso)
```

```
##   Degree APSE_Ridge_trend APSE_Ridge_trend_and_seasonality APSE_Elastic_trend
## 1      2         149.0067                         131.13430           164.4207
## 2      3         170.5809                          80.27174           152.5620
## 3      4         215.6233                         109.55550           154.7657
## 4      5         218.1155                          93.64583           155.5899
## 5      6         230.9763                         112.43128           155.5899
## 6      7         232.4437                          96.76718           156.4995
## 7      8         238.4260                         124.89896           156.4995
## 8      9         240.9129                         155.83503           157.4866
## 9     10         239.2895                         137.83042           157.4866
##   APSE_Elastic_trend_and_seasonality APSE_Lasso
## 1                          134.69736   152.3410
## 2                           76.14548   152.7167
## 3                           81.84240   152.7167
## 4                           80.98537   152.7167
## 5                           80.98537   152.7167
## 6                           80.98537   156.1483
## 7                           82.52566   152.7167
## 8                           80.78562   156.1483
## 9                           79.43380   156.1483
```

```
# Since we have multiple cases, we summarize all optimal cases for each model
# ridge regression
ridge.optimal.index = which.min(APSE.ridge)
```

```
ridge.both.optimal.index = which.min(APSE.ridge.both)
lambda.ridge.optimal = lambda.ridge[ridge.optimal.index]
lambda.ridge.both.optimal = lambda.ridge.both[ridge.both.optimal.index]
p.ridge.optimal = poly.degree[ridge.optimal.index]
p.ridge.both.optimal = poly.degree[ridge.both.optimal.index]

# elastic net regression
elastic.optimal.index = which.min(APSE.elastic)
elastic.both.optimal.index = which.min(APSE.elastic.both)
lambda.elastic.optimal = lambda.elastic[elastic.optimal.index]
lambda.elastic.both.optimal = lambda.elastic.both[elastic.both.optimal.index]
p.elastic.optimal = poly.degree[elastic.optimal.index]
p.elastic.both.optimal = poly.degree[elastic.both.optimal.index]

# lasso regression
lasso.optimal.index = which.min(APSE.lasso)
lambda.lasso.optimal = lambda.lasso[lasso.optimal.index]
p.lasso.optimal = poly.degree[lasso.optimal.index]

data.frame("Model" = c("Ridge_trend", "Ridge_trend_and_seasonality",
                       "Elastic_trend", "Elastic_trend_and_seasonality",
                       "Lasso_trend"),
           "Optimal_degree" = c(p.ridge.optimal, p.ridge.both.optimal,
                                p.elastic.optimal, p.elastic.both.optimal,
                                p.lasso.optimal),
           "APSE" = c(APSE.ridge[ridge.optimal.index],
                      APSE.ridge.both[ridge.both.optimal.index],
                      APSE.elastic[elastic.optimal.index],
                      APSE.elastic.both[elastic.both.optimal.index],
                      APSE.lasso[lasso.optimal.index]))
```

```
##                          Model Optimal_degree      APSE
## 1                  Ridge_trend              2 149.00671
## 2   Ridge_trend_and_seasonality              3  80.27174
## 3                Elastic_trend              3 152.56198
## 4 Elastic_trend_and_seasonality              3  76.14548
## 5                  Lasso_trend              2 152.34097
```

```
# We add seasonality to optimal lasso regression with trend model to see if seasonality
# improves the model
fit.train.lasso.both = glmnet(cbind(X[train.indx, 1:p.lasso.optimal], Seasonmatrix[train.indx,]),
                              as.vector(wheatflourTS.train), alpha = 1,
                              lambda = lambda.lasso.optimal,
                              standardize = TRUE, intercept = TRUE,
                              family = "gaussian")
Fitted.test.lasso.both = predict(fit.train.lasso.both, newx = cbind(X[-train.indx, 1:p.lasso.optimal],
                                                                    Seasonmatrix[-train.indx,]),
                                 type = "response")
APSE.lasso.both = mean((wheatflourTS.test - Fitted.test.lasso.both)^2)
# APSE is smaller for lasso, so adding seasonality indeed get a better model for prediction
# but still larger than the elastic net model
APSE.lasso.both
```

```
## [1] 108.4187
```

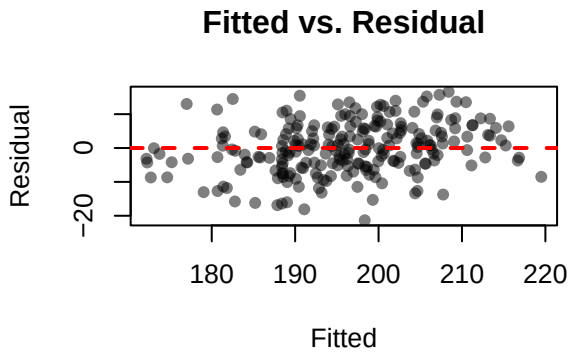
```r
# the seasonal components are not all included (do not fit the model with seasonal component)
fit.train.lasso.both$beta
```

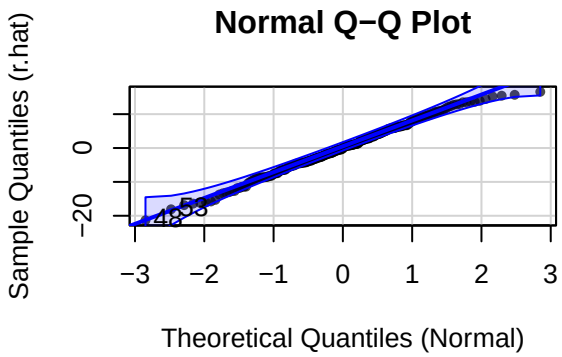
```
## 13 x 1 sparse Matrix of class "dgCMatrix"
##                    s0
## 1         32.6018404
## 2         73.1050399
## Season2  -13.2368665
## Season3    .
## Season4   -4.4837380
## Season5    .
## Season6    .
## Season7    .
## Season8    1.0515283
## Season9    0.6617759
## Season10   5.2872987
## Season11   2.9106026
## Season12  -3.5765872
```

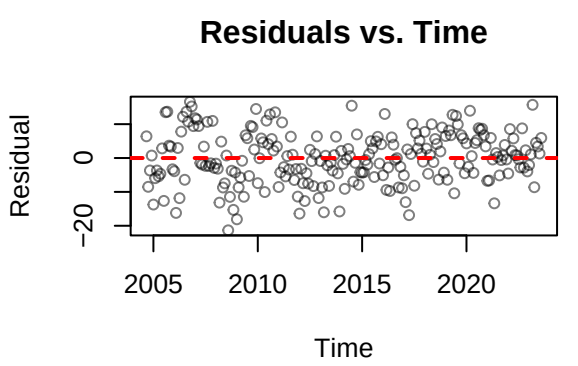
```r
# We choose the trend + seasonality Elastic Net regression model with p = 3
# We fit the optimal model to the training data
p.optimal = p.elastic.both.optimal
lambda.optimal = lambda.elastic.both.optimal
fit.optimal = glmnet(cbind(X[train.indx, 1:p.optimal],
                           Seasonmatrix[train.indx,]), as.vector(wheatflourTS.train),
                     alpha = 0.5,
                     lambda = lambda.optimal,
                     standardize = TRUE, intercept = TRUE,
                     family = "gaussian")

# Perform model diagnostics to check the fit performance
par(mfrow = c(2, 2))
fitted = predict(fit.optimal, newx = cbind(X[train.indx, 1:p.optimal],
                                           Seasonmatrix[train.indx,]), type = "response")
residuals = wheatflourTS.train - fitted
plot(fitted, residuals, pch = 16, col = adjustcolor("black", 0.5),
     xlab = "Fitted", ylab = "Residual", main = "Fitted vs. Residual")
abline(h = 0, lty = 2, lwd = 2, col = "red")
car::qqPlot(residuals, pch = 16, col = adjustcolor("black", 0.7),
            xlab = "Theoretical Quantiles (Normal)", ylab = "Sample Quantiles (r.hat)",
            main = "Normal Q-Q Plot")
```

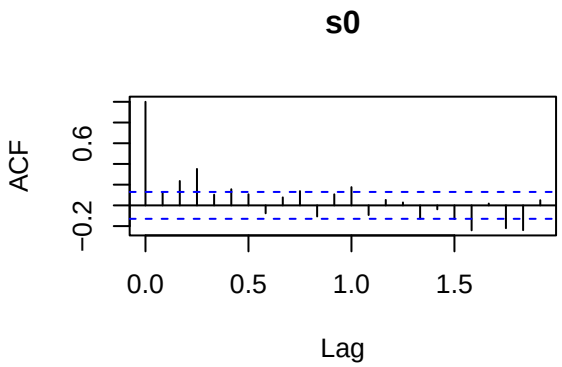
```
## [1] 48 53
```

```r
plot(residuals, type = "p", col = adjustcolor("black", 0.5),
     ylab = "Residual", main = "Residuals vs. Time")
abline(h = 0, lty = 2, lwd = 2, col = "red")
acf(residuals)
```

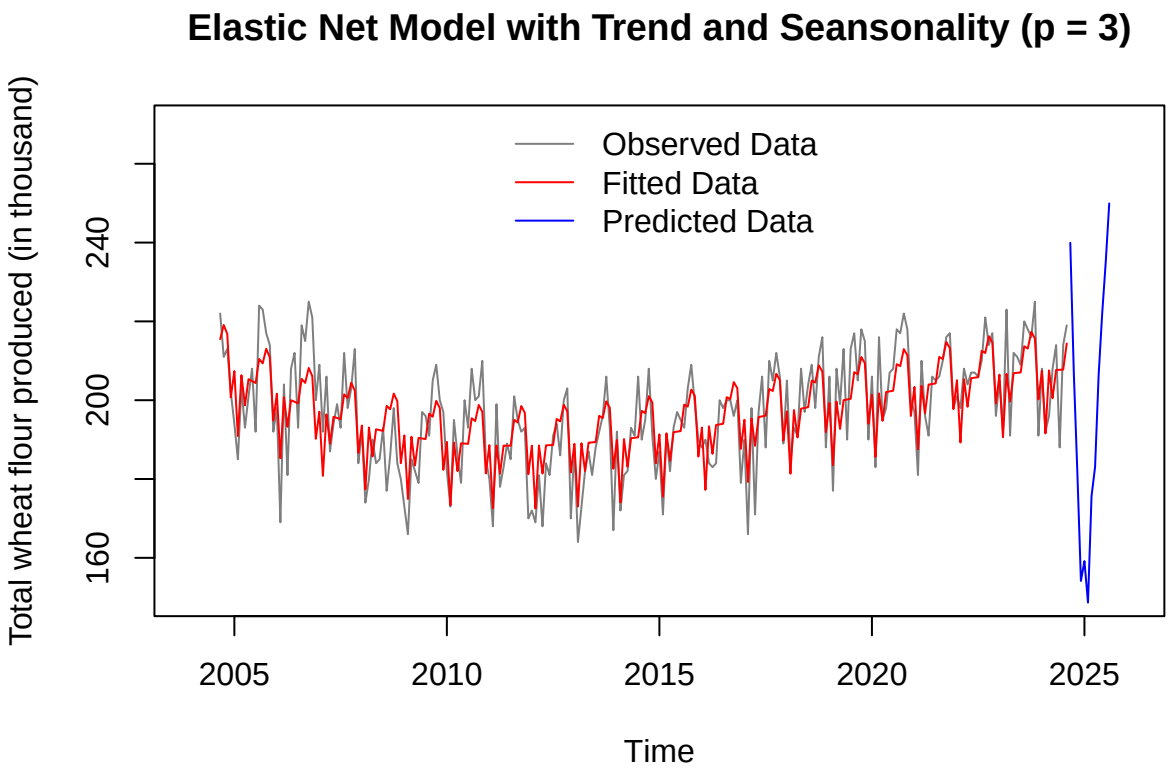









```r
# Plot the observed data, fitted data, and predicted data
model.regularization = glmnet(cbind(X[, 1:p.optimal], Seasonmatrix), as.vector(wheatflourTS),
                              alpha = 0.5,
                              lambda = lambda.optimal,
                              standardize = TRUE, intercept = TRUE,
                              family = "gaussian") # final model
Fitted.regularization = predict(model.regularization, newx = cbind(X[, 1:p.optimal], Seasonmatrix),
                                type = "response")
predict.time = seq(2024 + 8/12, 2025 + 7/12, length = 12)
Seasonmatrix.predict = model.matrix(~ as.factor(c(9:12, 1:8)))[, 2:12]
X.predict = cbind(poly(predict.time, p.optimal), Seasonmatrix.predict)
Fitted.predict.regularization = predict(model.regularization, newx = X.predict, type = "response")
plot(wheatflourTS, xlim = c(2004, 2026), ylim = c(150, 270), col = adjustcolor("black", 0.5),
     ylab = "Total wheat flour produced (in thousand)",
     main = "Elastic Net Model with Trend and Seansonality (p = 3)") # observed data
lines(time, Fitted.regularization, col = "red") # fitted data
lines(predict.time, Fitted.predict.regularization, col = "blue") # predicted data
legend("top", legend = c("Observed Data", "Fitted Data", "Predicted Data"),
       col = c(adjustcolor("black", 0.5), "red", "blue"), lty = 1, bty = "n")
```



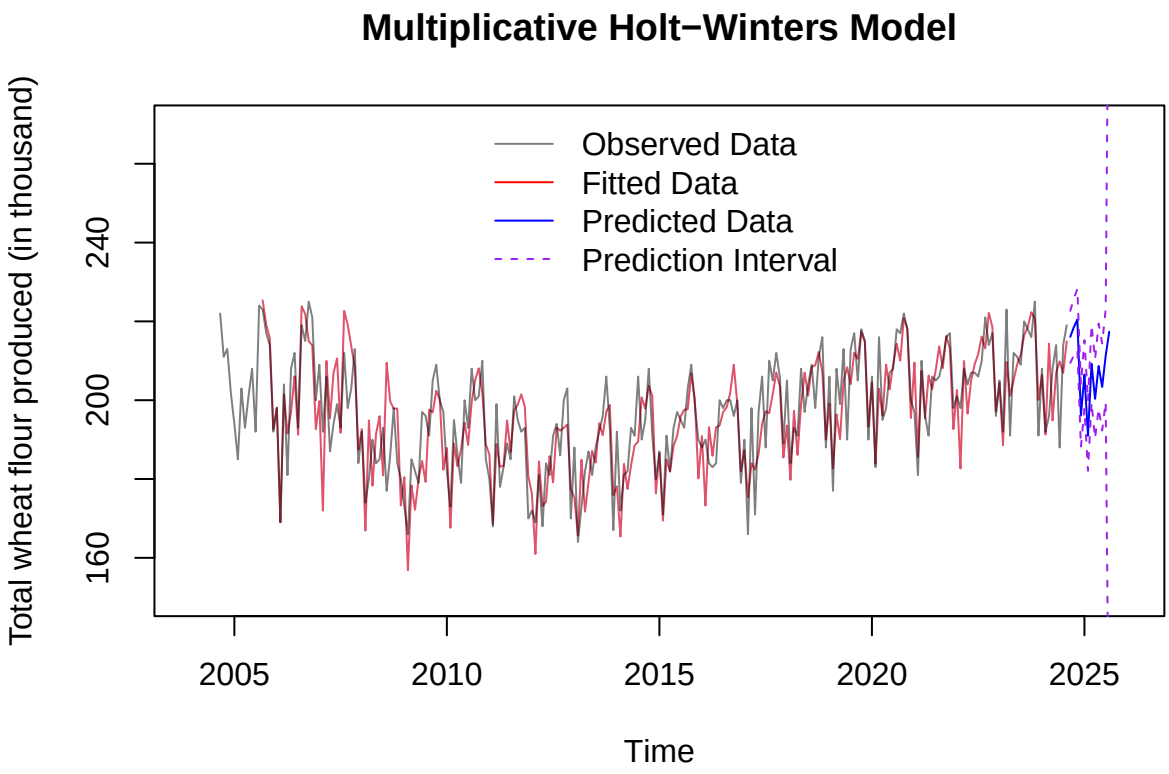
Holt-Winter Model

```
# Next we check the Holt-Winter model
# Fit all 4 cases of Holt-Winter model
# simple exponential smoothing
es <- HoltWinters(wheatflourTS.train, gamma=FALSE , beta=FALSE)
es.predict = predict(es, n.ahead=12)

# double exponential smoothing
hw <- HoltWinters(wheatflourTS.train, gamma=FALSE)
hw.predict = predict(hw, n.ahead=12)

# additive Holt-Winters
hw.additive <- HoltWinters(wheatflourTS.train, seasonal="additive")
hw.additive.predict = predict(hw.additive, n.ahead=12)

# multiplicative Holt-Winters
hw.multiplicative <- HoltWinters(wheatflourTS.train, seasonal="multiplicative")
hw.multiplicative.predict = predict(hw.multiplicative, n.ahead=12)

# Calculate the APSE value for each model
APSE.es = mean((wheatflourTS.test-es.predict)^2)
APSE.hw = mean((wheatflourTS.test-hw.predict)^2)
APSE.hw.additive = mean((wheatflourTS.test-hw.additive.predict)^2)
```

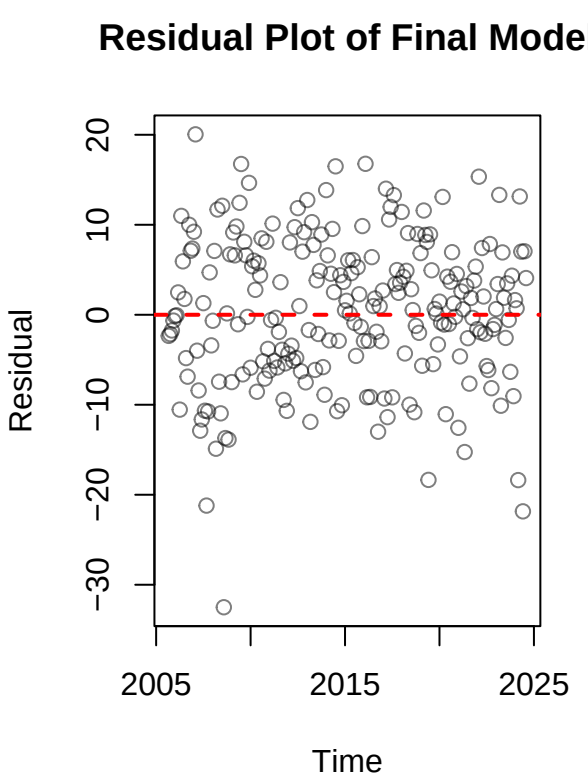
```r
APSE.hw.multiplicative = mean((wheatflourTS.test-hw.multiplicative.predict)^2)
APSE = c(APSE.es, APSE.hw, APSE.hw.additive, APSE.hw.multiplicative)
data.frame(Model = c("Simple Exponential Smoothing",
                     "Double Exponential Smoothing",
                     "Additive Holt-Winters",
                     "Multiplicative Holt-Winters"),
           APSE=APSE)
```

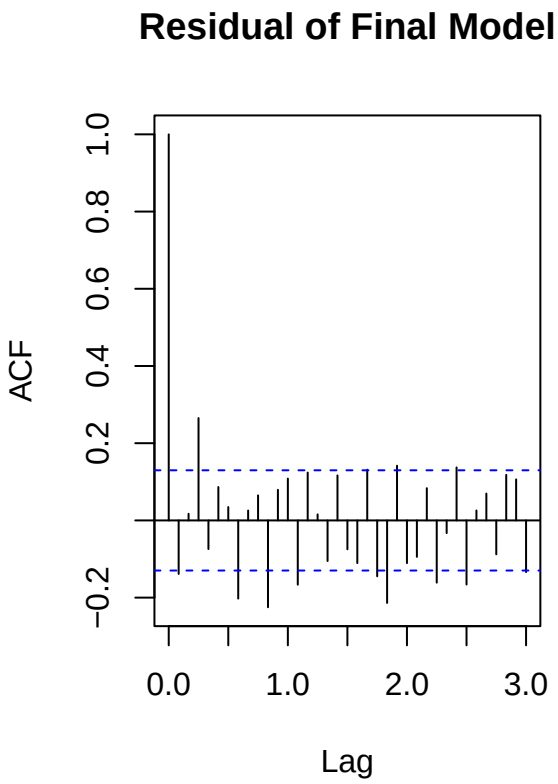
```
##                           Model     APSE
## 1 Simple Exponential Smoothing 167.4360
## 2 Double Exponential Smoothing 308.2096
## 3        Additive Holt-Winters 103.2490
## 4  Multiplicative Holt-Winters 101.1752
```

```r
# We choose the Multiplicative Holt-Winters model
# We fit the optimal model to the training data
final.fit.hw <- HoltWinters(wheatflourTS, seasonal="multiplicative")
fit.hw.multiplicative = predict(final.fit.hw, n.ahead=12, prediction.interval=TRUE)

# Plot the observed data, fitted data, predicted data, and 95% prediction interval
plot(final.fit.hw, xlim=c(2004, 2026), ylim=c(150, 270),
     col=adjustcolor("black", 0.5), ylab="Total wheat flour produced (in thousand)",
     main="Multiplicative Holt-Winters Model") # observed data + fitted data
lines(fit.hw.multiplicative[, 1], col="blue") # predicted data
lines(fit.hw.multiplicative[, 2] ~ predict$Time, lty=2, col="purple") # 95% prediction interval
lines(fit.hw.multiplicative[, 3] ~ predict$Time, lty=2, col="purple") # 95% prediction interval
legend("top", legend=c("Observed Data", "Fitted Data", "Predicted Data", "Prediction Interval"),
       col=c(adjustcolor("black", 0.5), "red", "blue", "purple"),
       lty=c(1, 1, 1, 2), bty="n")
```



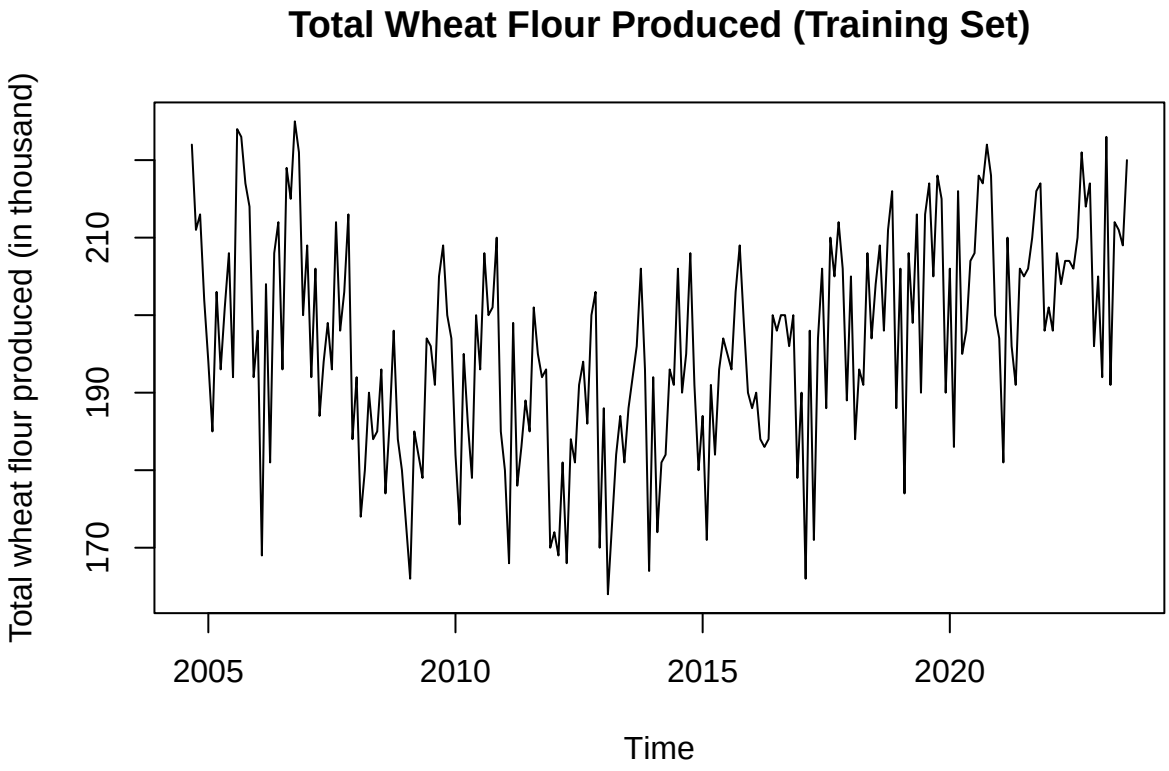

```r
# We can plot the residual plots to check the stationarity after Holt-Winter model
par(mfrow=c(1, 2))
plot(residuals(final.fit.hw), type = "p", ylab="Residual",
     main="Residual Plot of Final Model", , col = adjustcolor("black", 0.5))
abline(h = 0, lty = 2, lwd = 2, col = "red")
acf(residuals(final.fit.hw), lag.max=36, main="Residual of Final Model")
```



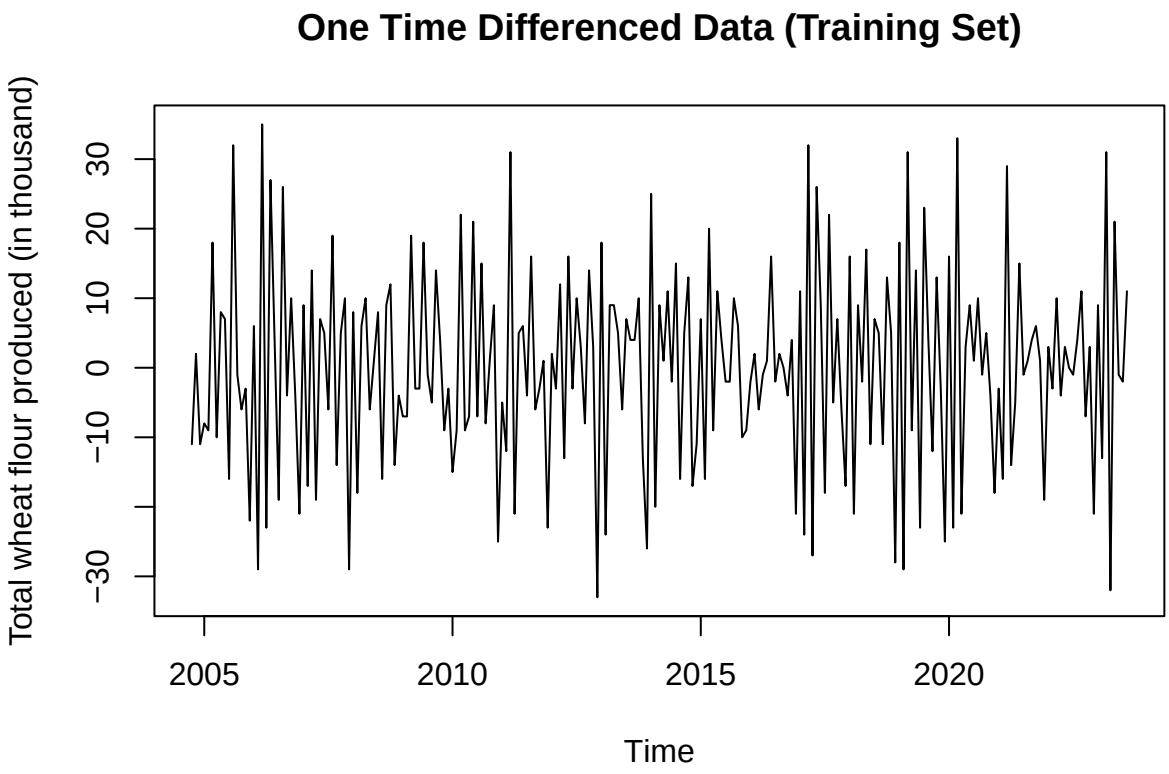


Box-Jenkins Model

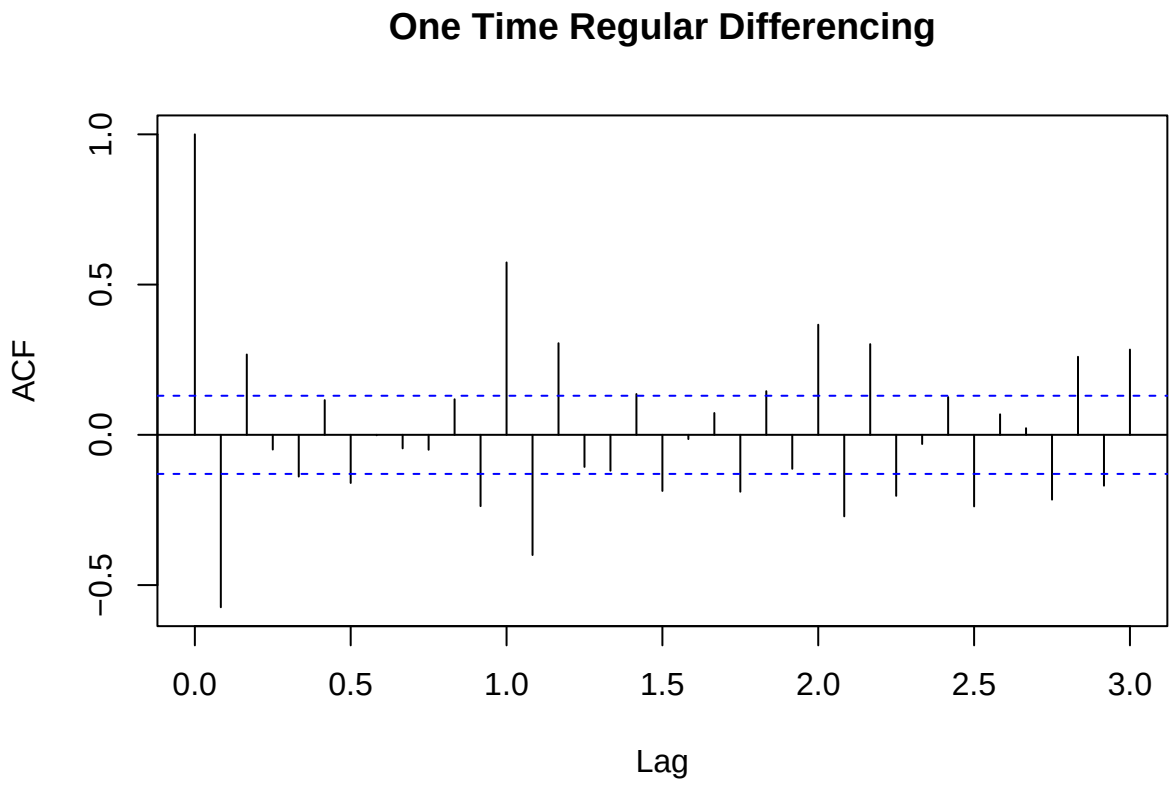
```
# non-stationary, has trend and seasonality
plot(wheatflourTS.train, ylab="Total wheat flour produced (in thousand)",
     main="Total Wheat Flour Produced (Training Set)")
```



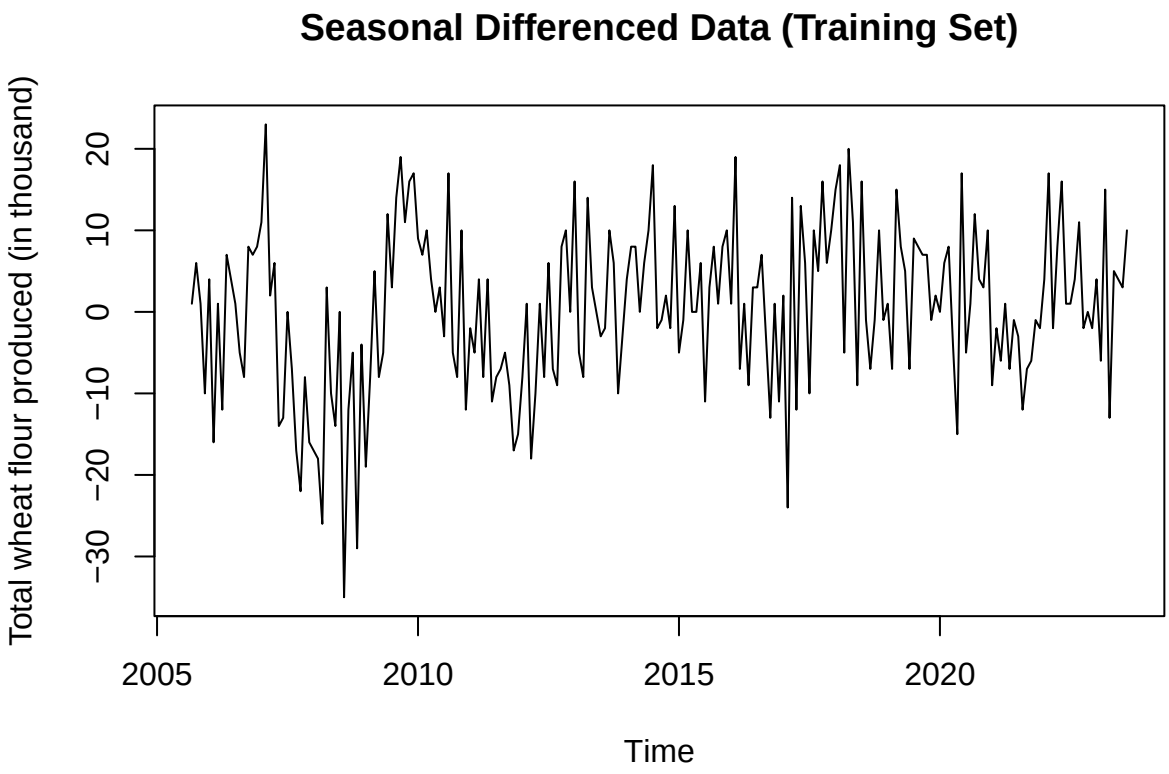

```
diff_1 <- diff(wheatflourTS.train, differences=1)
plot(diff_1, ylab="Total wheat flour produced (in thousand)",
     main="One Time Differenced Data (Training Set)")
```



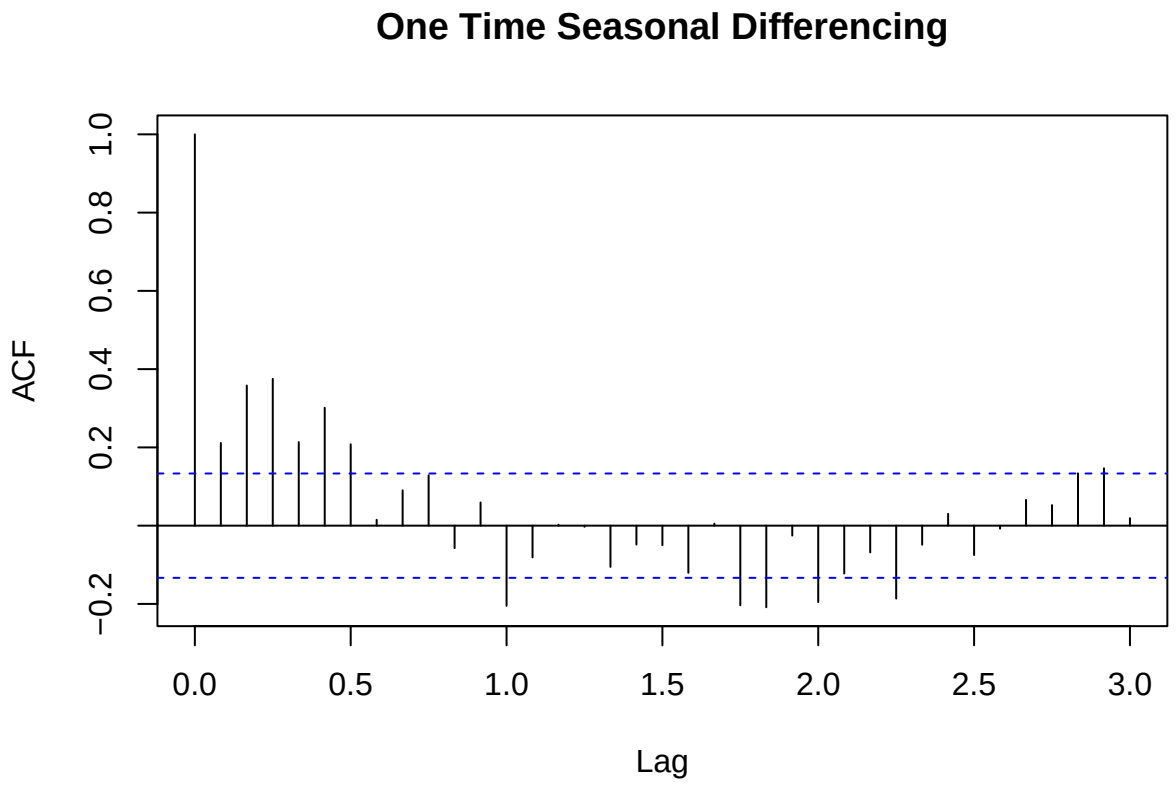

```
# non-stationary, has correlation on seasonal lag
acf(diff_1, lag.max=36, main="One Time Regular Differencing")
```



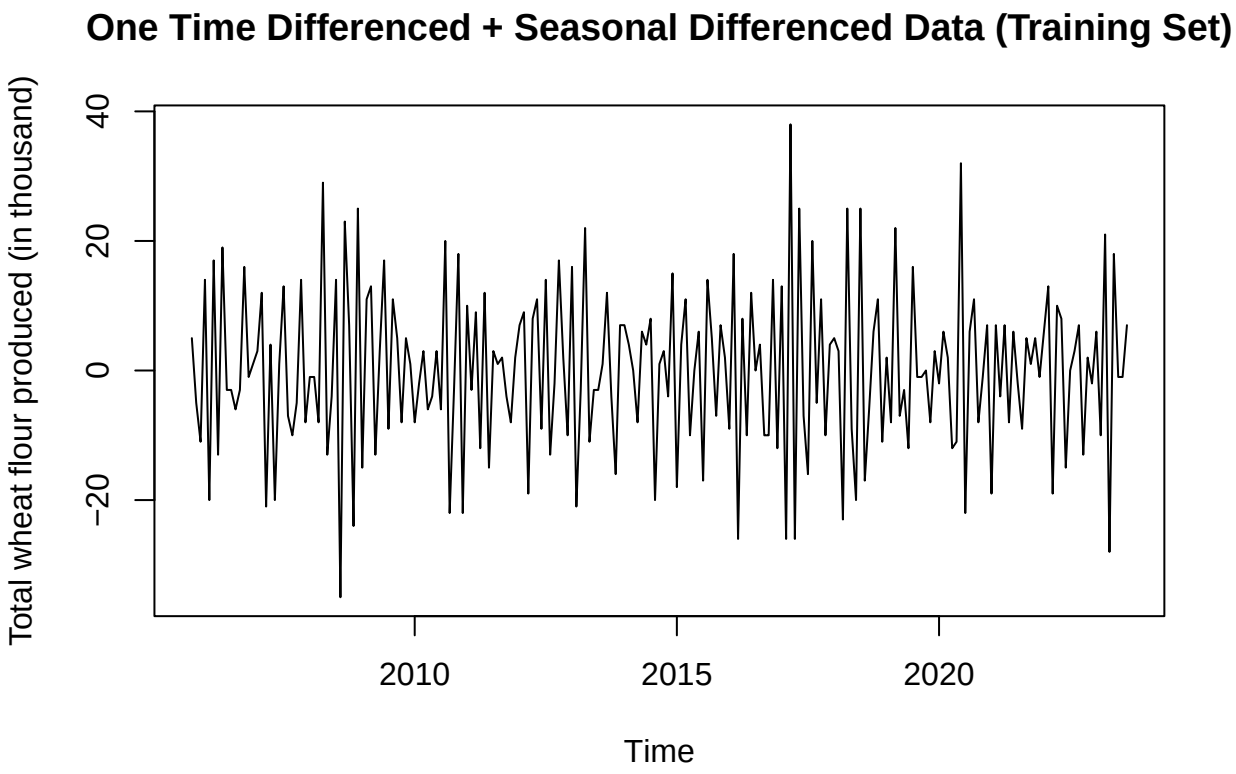

```
diff_s <- diff(wheatflourTS.train, lag=12)
plot(diff_s, ylab="Total wheat flour produced (in thousand)",
     main="Seasonal Differenced Data (Training Set)")
```



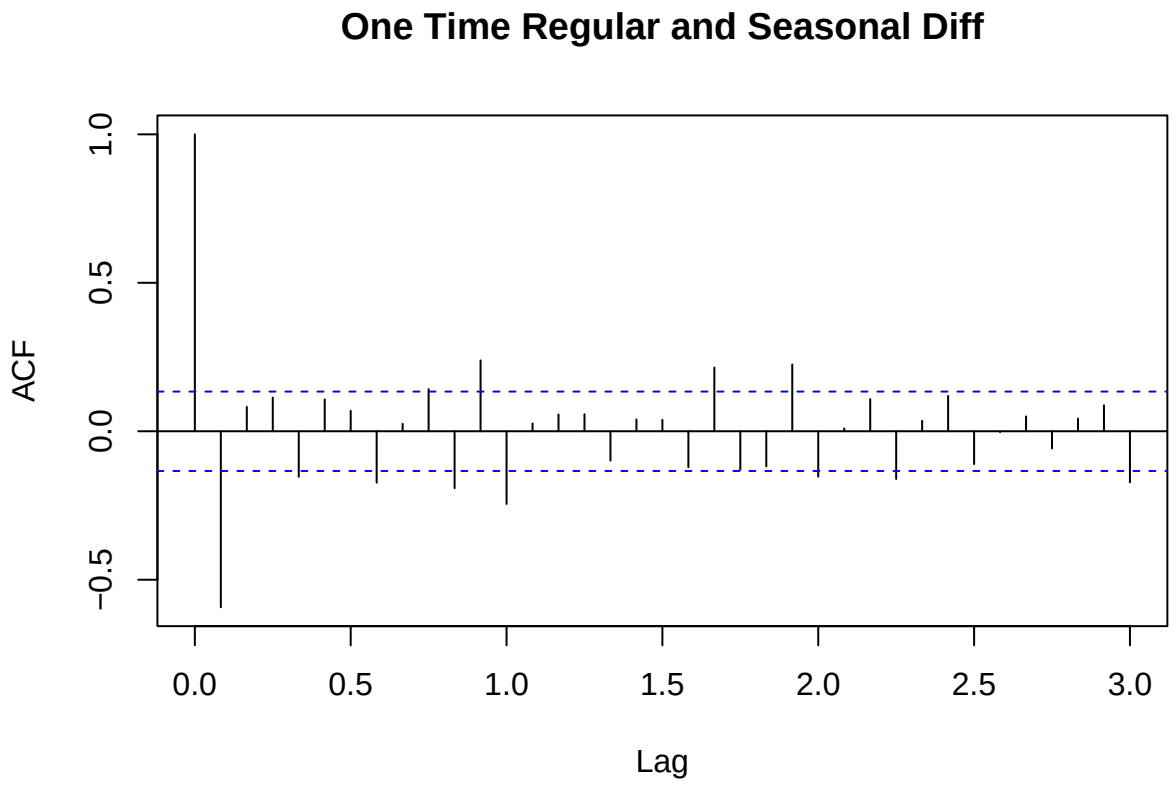

```r
# non-stationary since lag 12 and lag 24 pop out
acf(diff_s, lag.max=36, main="One Time Seasonal Differencing")
```



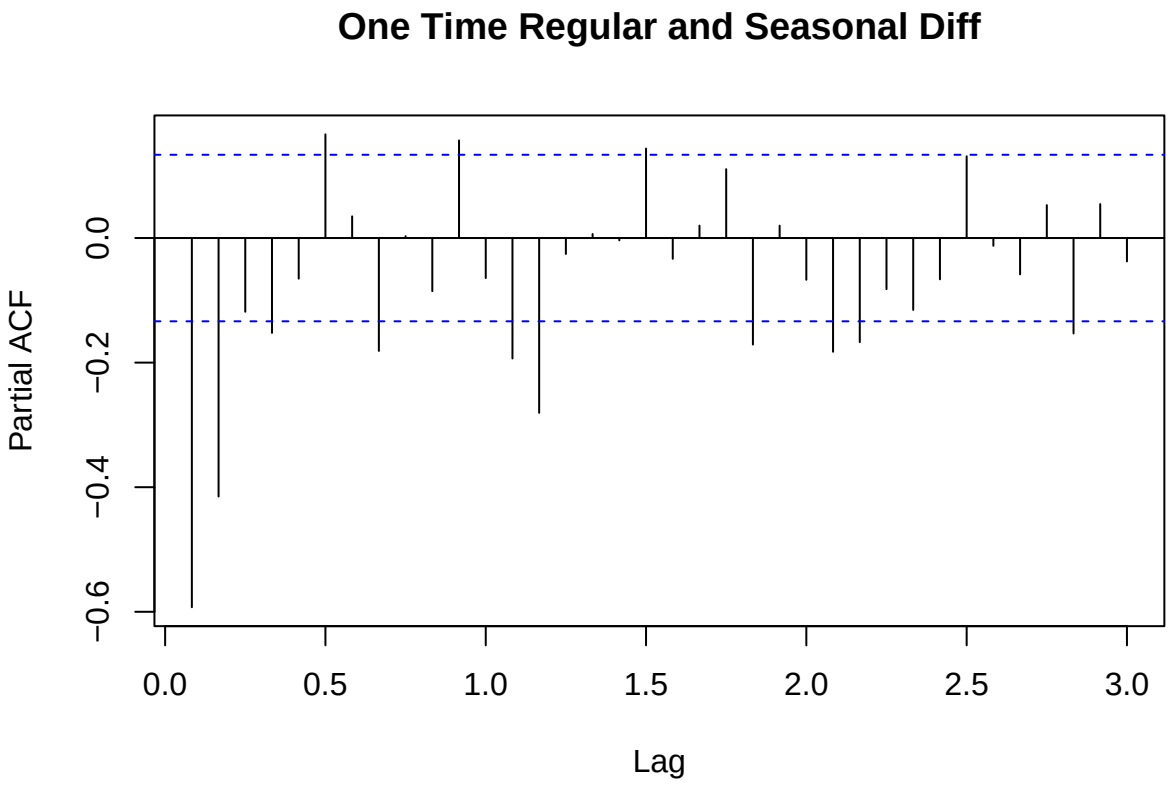

```
diff_final <- diff(diff_s, differences=1)
plot(diff_final, ylab="Total wheat flour produced (in thousand)",
     main="One Time Differenced + Seasonal Differenced Data (Training Set)")
```




```
# lag 12 still pops out, but OK
acf(diff_final, lag.max=36, main="One Time Regular and Seasonal Diff")
```




```
# In SARIMA, d = D = 1, s = 12
pacf(diff_final, lag.max=36, main="One Time Regular and Seasonal Diff")
```



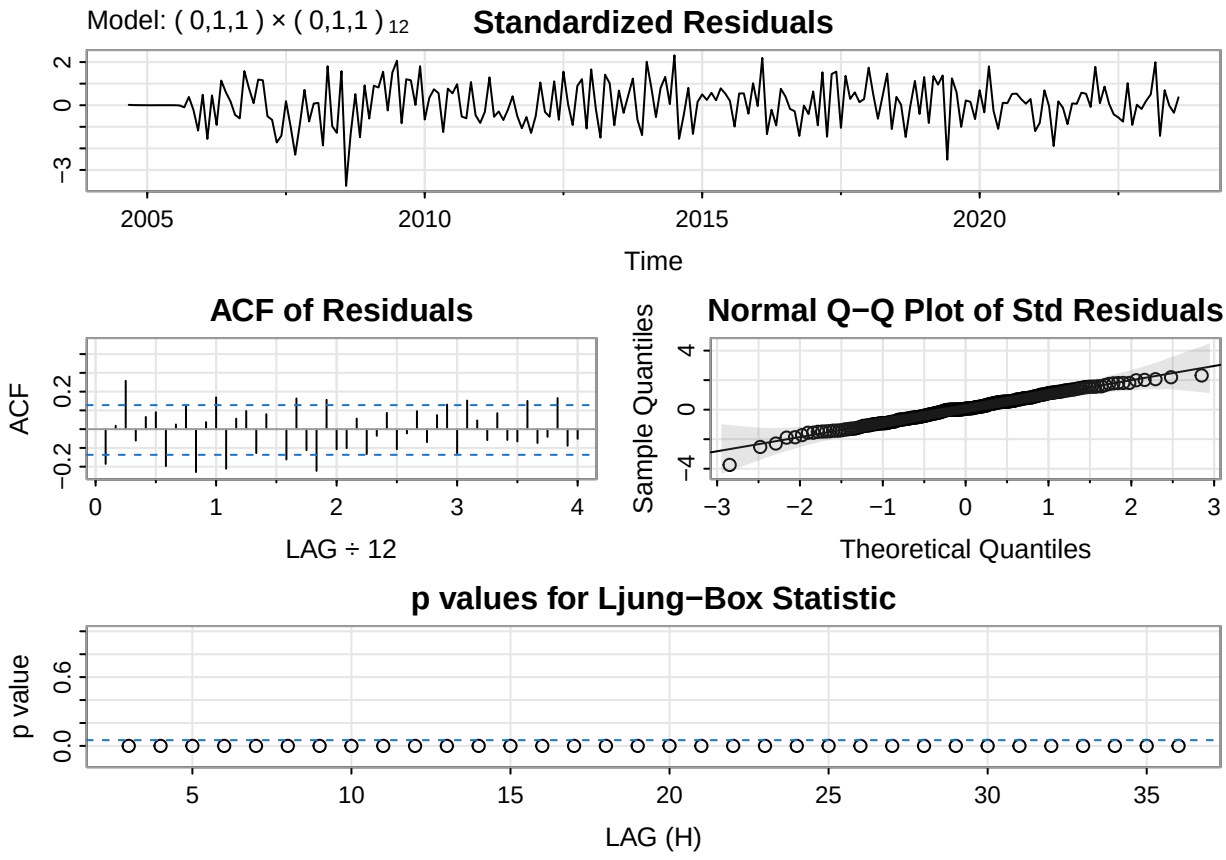

```
# We propose the following models.
library(astsa)
```

```
## Warning: package 'astsa' was built under R version 4.5.3
```

```
fit1 <- sarima(wheatflourTS.train, p=0,d=1,q=1,P=0,D=1,Q=1,S=12)
```

```
## initial  value 2.523091
## iter   2 value 2.196753
## iter   3 value 2.151510
## iter   4 value 2.113082
## iter   5 value 2.086249
## iter   6 value 2.084701
## iter   7 value 2.083212
## iter   8 value 2.082844
## iter   9 value 2.082824
## iter  10 value 2.082818
## iter  11 value 2.082817
## iter  11 value 2.082817
## final  value 2.082817
## converged
## initial  value 2.080033
## iter   2 value 2.071057
## iter   3 value 2.070454
## iter   4 value 2.070236
```

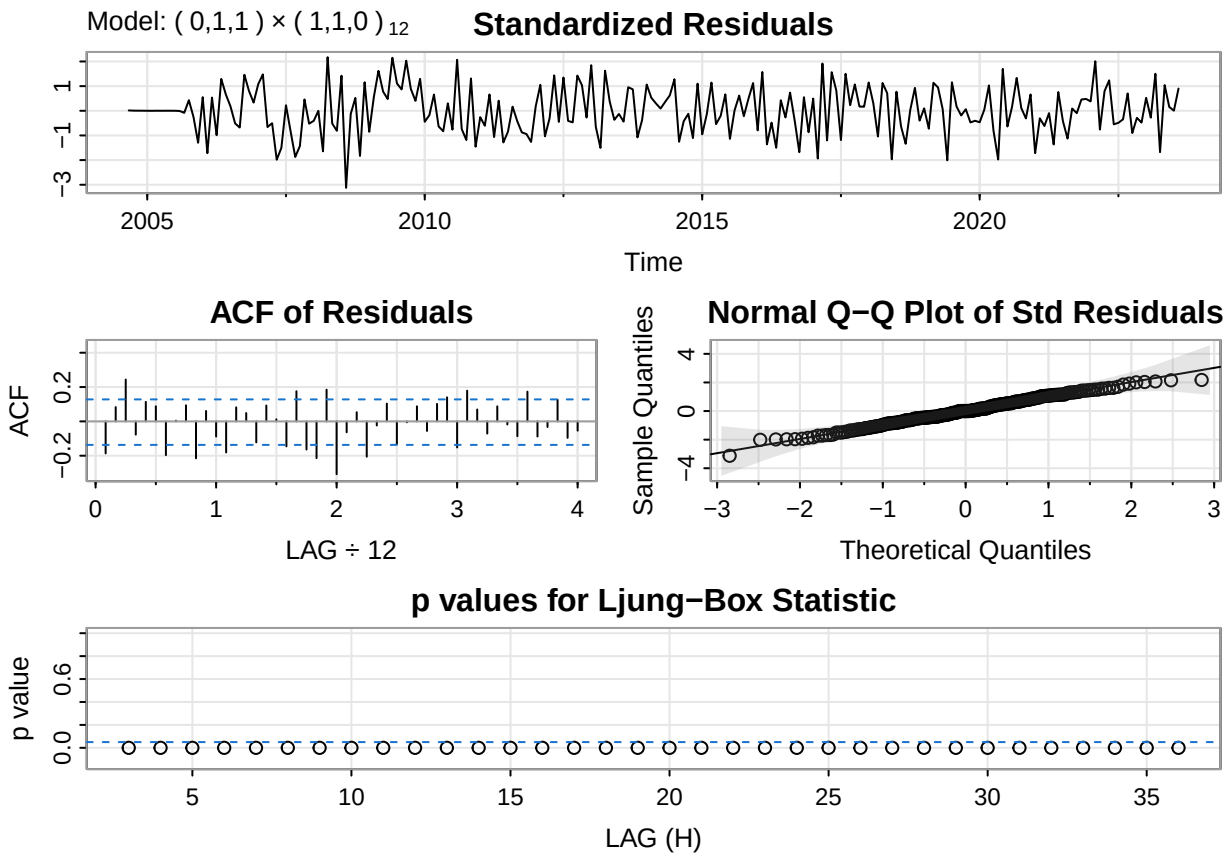
```
## iter   5 value 2.070235
## iter   5 value 2.070235
## iter   5 value 2.070235
## final  value 2.070235
## converged
## <><><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ma1   -0.7877 0.0382 -20.6364       0
## sma1  -0.9115 0.0833 -10.9380       0
##
## sigma^2 estimated as 56.7138 on 213 degrees of freedom
##
## AIC = 7.006255  AICc = 7.006518  BIC = 7.053287
##
```




```
fit2 <- sarima(wheatflourTS.train, p=0,d=1,q=1,P=1,D=1,Q=0,S=12)
```

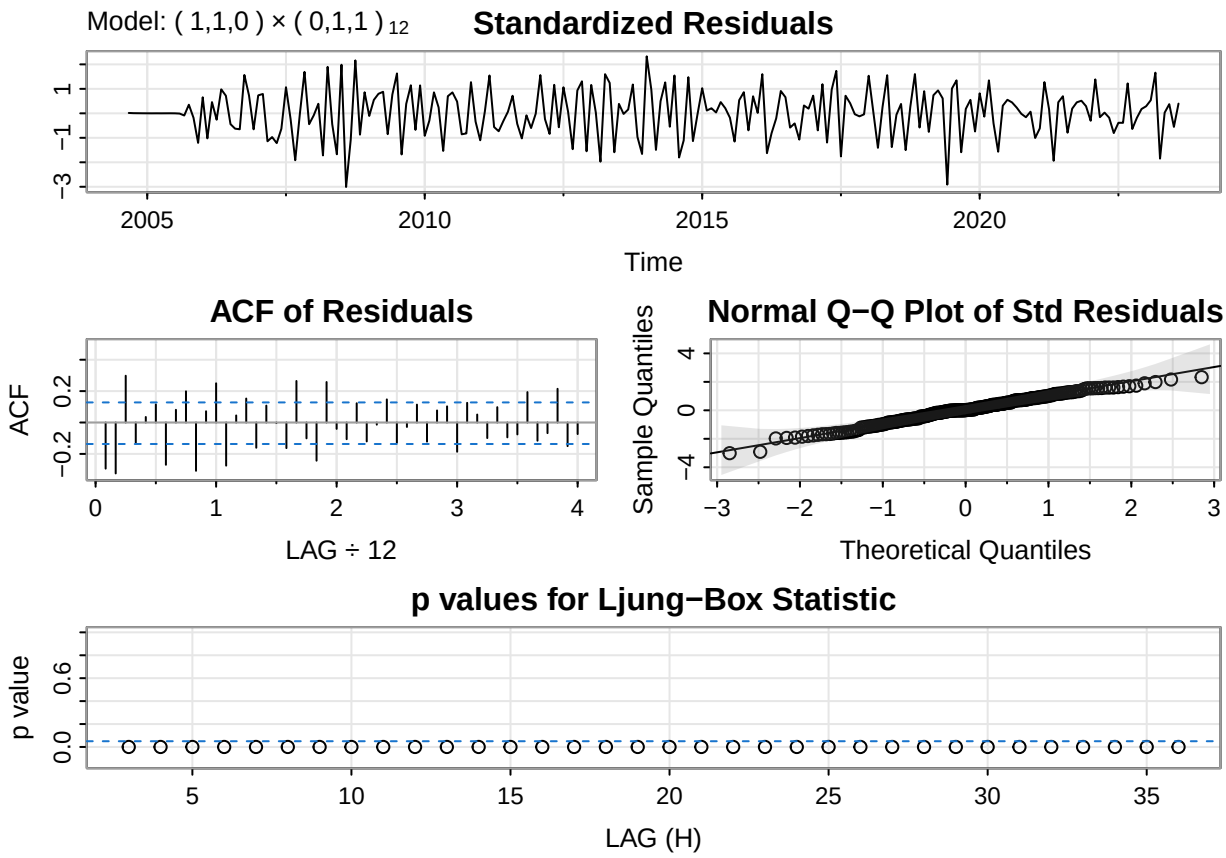
```
## initial  value 2.526504
## iter   2 value 2.220381
## iter   3 value 2.197504
## iter   4 value 2.193439
## iter   5 value 2.191896
## iter   6 value 2.191843
```

```
## iter   7 value 2.191843
## iter   8 value 2.191843
## iter   8 value 2.191843
## iter   8 value 2.191843
## final  value 2.191843
## converged
## initial  value 2.182610
## iter   2 value 2.182534
## iter   3 value 2.182503
## iter   4 value 2.182503
## iter   4 value 2.182503
## iter   4 value 2.182503
## final  value 2.182503
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ma1   -0.7622 0.0418 -18.2499       0
## sar1  -0.3091 0.0655  -4.7193       0
##
## sigma^2 estimated as 77.88602 on 213 degrees of freedom
##
## AIC = 7.230791  AICc = 7.231054  BIC = 7.277823
##
```



```r
fit3 <- sarima(wheatflourTS.train, p=1,d=1,q=0,P=0,D=1,Q=1,S=12)
```

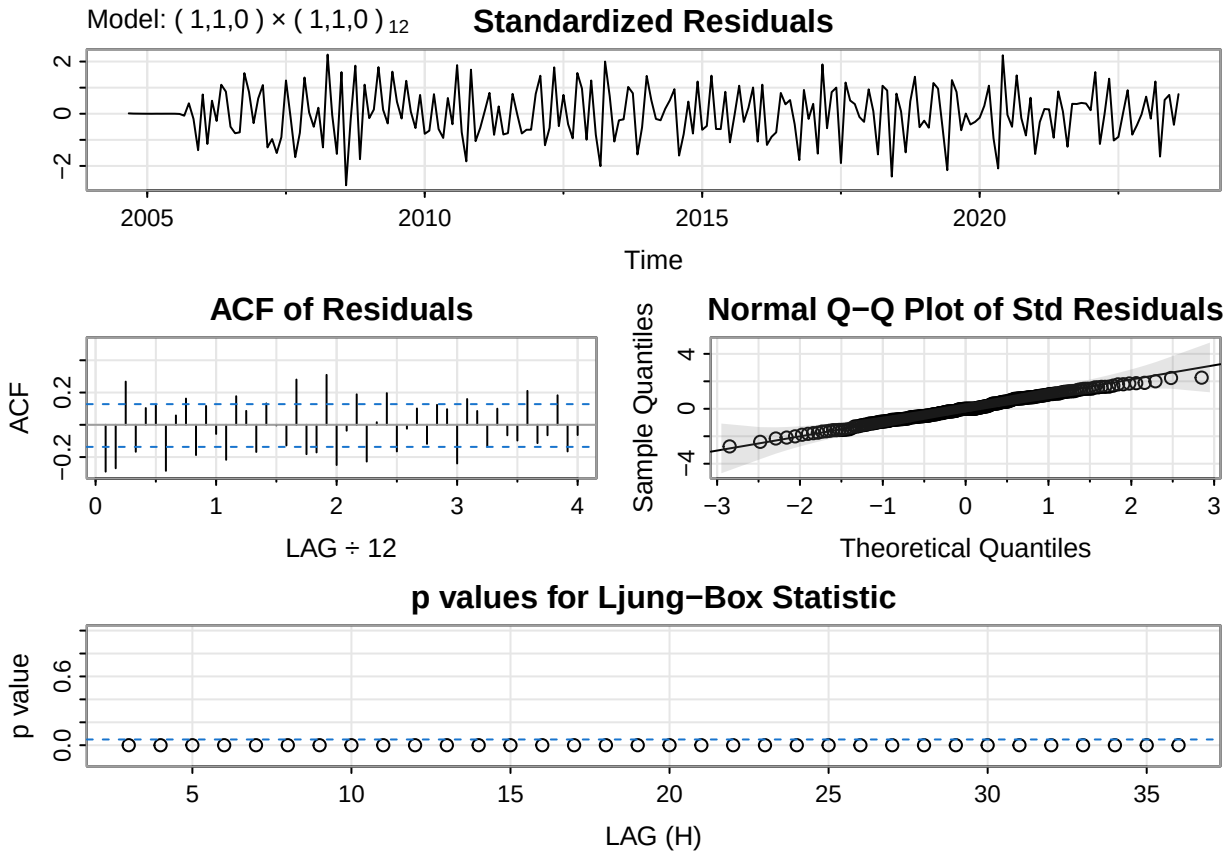
```
## initial  value 2.525048
## iter   2 value 2.255361
## iter   3 value 2.227369
## iter   4 value 2.212034
## iter   5 value 2.204812
## iter   6 value 2.203980
## iter   7 value 2.203126
## iter   8 value 2.202298
## iter   9 value 2.202244
## iter  10 value 2.202232
## iter  11 value 2.202232
## iter  11 value 2.202232
## final  value 2.202232
## converged
## initial  value 2.197720
## iter   2 value 2.188231
## iter   3 value 2.185817
## iter   4 value 2.181749
## iter   5 value 2.181580
## iter   6 value 2.181575
## iter   7 value 2.181575
## iter   8 value 2.181574
## iter   8 value 2.181574
## iter   8 value 2.181574
## final  value 2.181574
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ar1   -0.5845 0.0556 -10.5184       0
## sma1  -1.0000 0.1474  -6.7827       0
##
## sigma^2 estimated as 66.49534 on 213 degrees of freedom
##
## AIC = 7.228933  AICc = 7.229196  BIC = 7.275965
##
```




```
fit4 <- sarima(wheatflourTS.train, p=1,d=1,q=0,P=1,D=1,Q=0,S=12)
```

```
## initial  value 2.524928
## iter   2 value 2.272256
## iter   3 value 2.271978
## iter   4 value 2.271972
## iter   5 value 2.271971
## iter   5 value 2.271971
## iter   5 value 2.271971
## final  value 2.271971
## converged
## initial  value 2.268146
## iter   2 value 2.268070
## iter   3 value 2.268066
## iter   4 value 2.268065
## iter   4 value 2.268065
## iter   4 value 2.268065
## final  value 2.268065
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ar1   -0.5991 0.0545 -10.9976       0
## sar1  -0.2769 0.0659  -4.2023       0
```

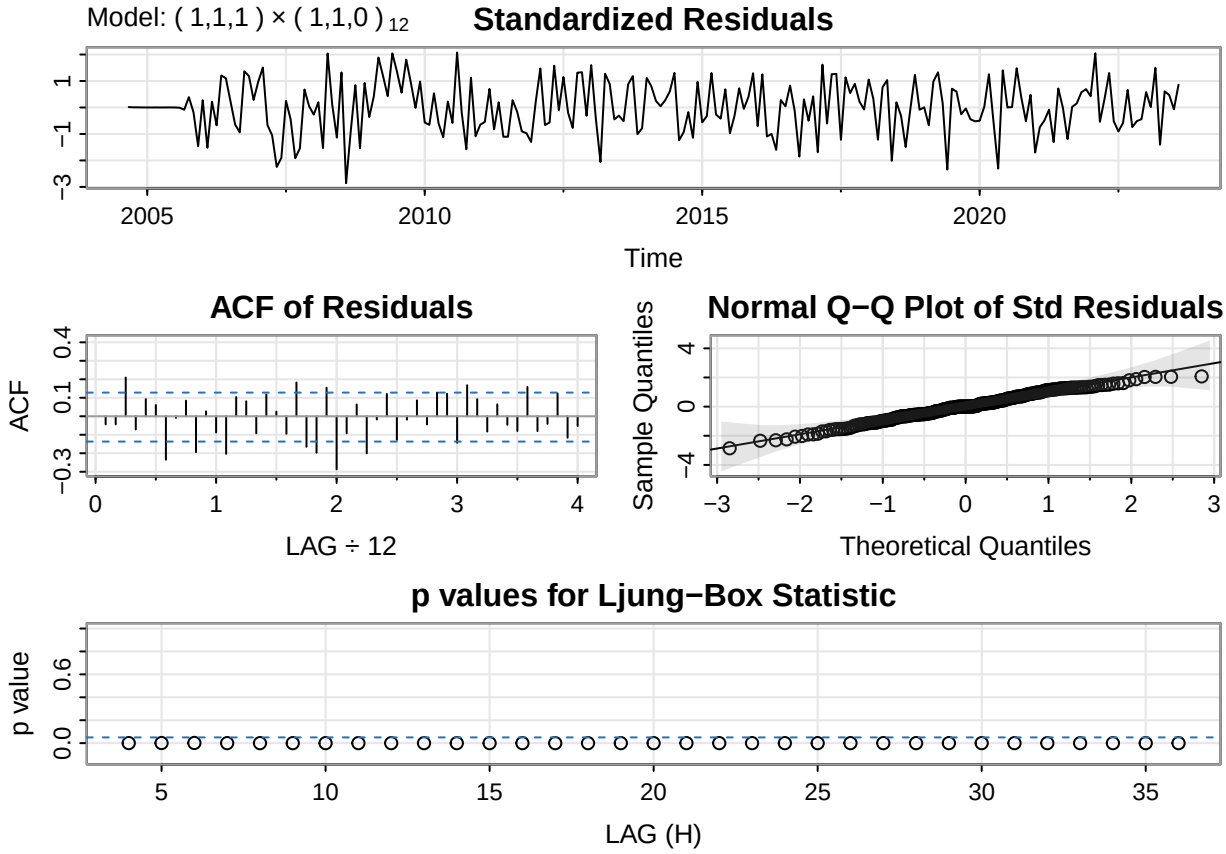
```
##
## sigma^2 estimated as 92.72304 on 213 degrees of freedom
##
## AIC = 7.401915  AICc = 7.402178  BIC = 7.448947
##
```




```
fit5 <- sarima(wheatflourTS.train, p=1,d=1,q=1,P=1,D=1,Q=0,S=12)
```

```
## initial  value 2.524928
## iter   2 value 2.181501
## iter   3 value 2.150557
## iter   4 value 2.143843
## iter   5 value 2.143209
## iter   6 value 2.143193
## iter   7 value 2.143193
## iter   7 value 2.143193
## iter   7 value 2.143193
## final  value 2.143193
## converged
## initial  value 2.148297
## iter   2 value 2.148287
## iter   3 value 2.148285
## iter   4 value 2.148285
## iter   4 value 2.148285
## iter   4 value 2.148285
```

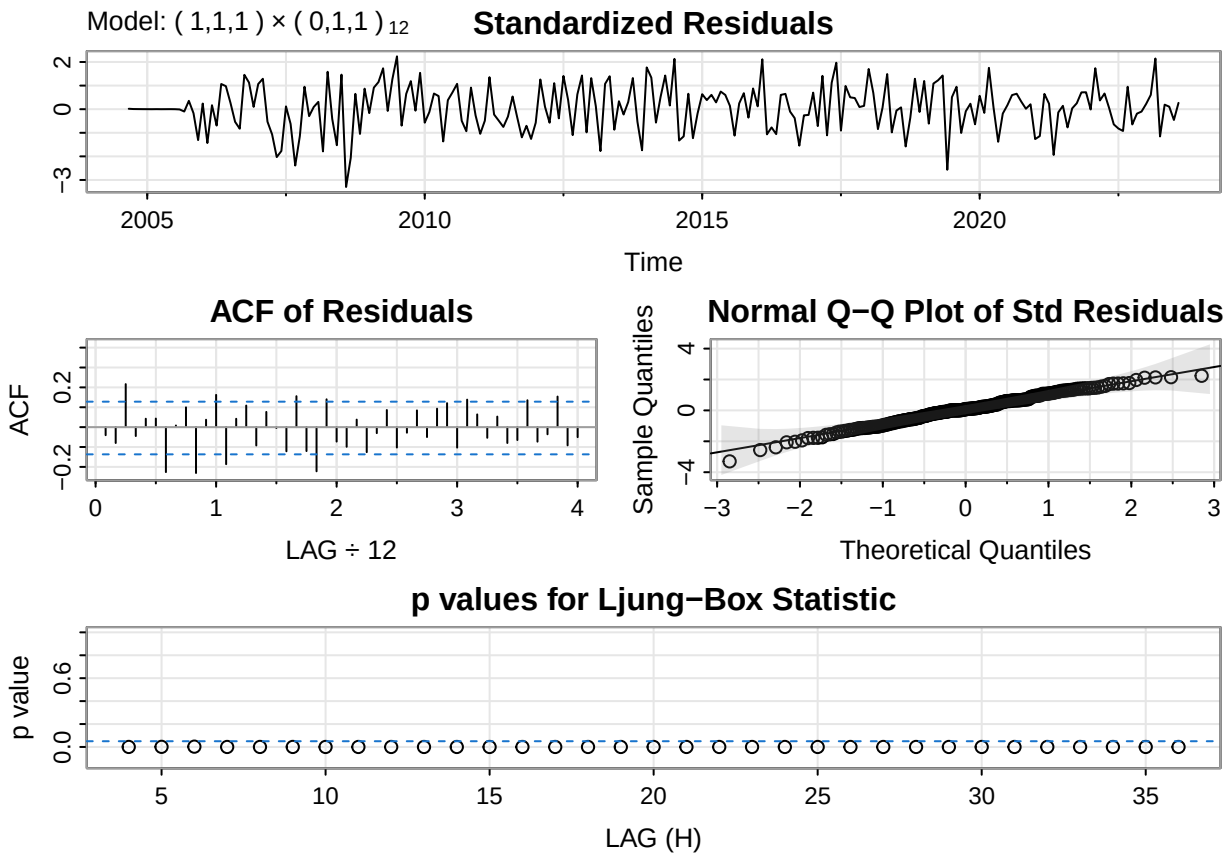
```
## final  value 2.148285
## converged
## <><><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE t.value p.value
## ar1   -0.3334 0.0792 -4.2099       0
## ma1   -0.6088 0.0622 -9.7817       0
## sar1  -0.3384 0.0650 -5.2103       0
##
## sigma^2 estimated as 72.62809 on 212 degrees of freedom
##
## AIC = 7.171656  AICc = 7.172185  BIC = 7.234365
##
```




```
fit6 <- sarima(wheatflourTS.train, p=1,d=1,q=1,P=0,D=1,Q=1,S=12)
```

```
## initial  value 2.525048
## iter   2 value 2.176276
## iter   3 value 2.098705
## iter   4 value 2.078342
## iter   5 value 2.062958
## iter   6 value 2.059696
## iter   7 value 2.056960
## iter   8 value 2.056593
```

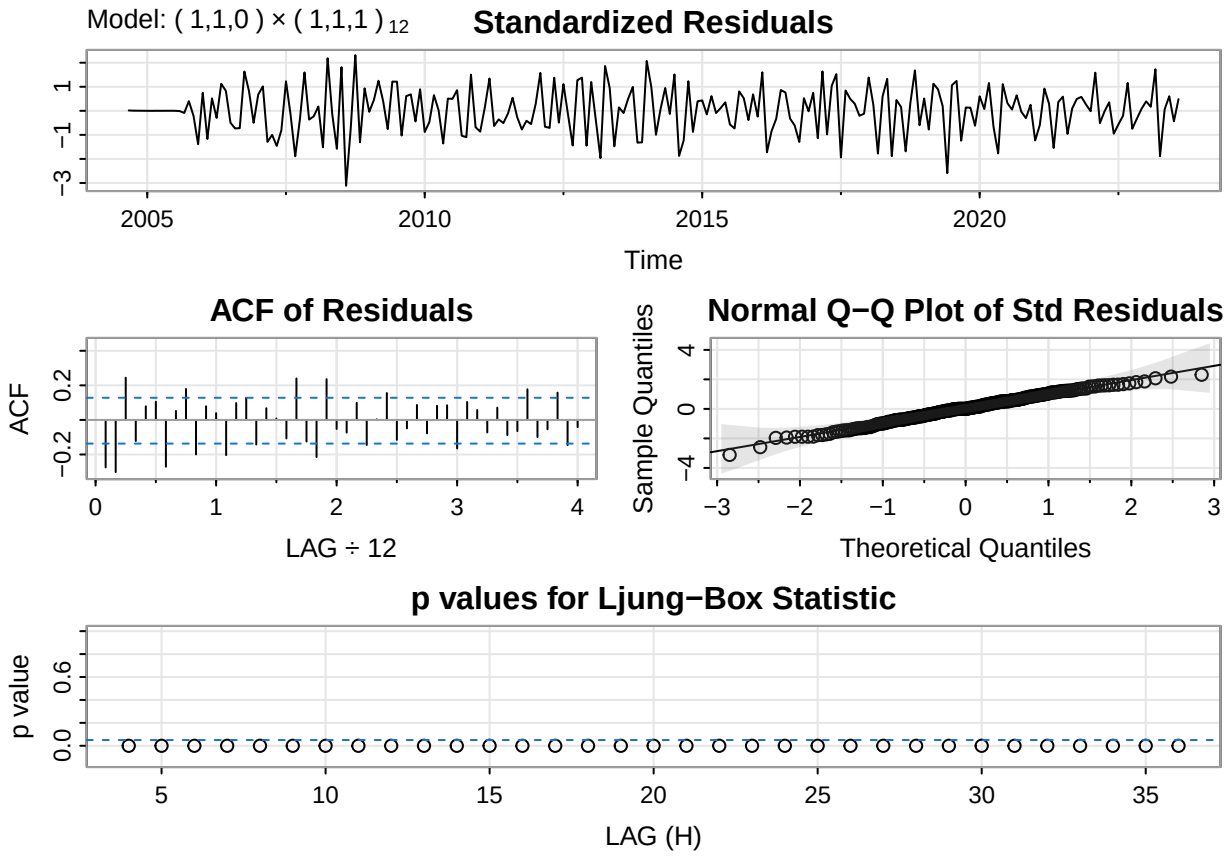
```
## iter   9 value 2.056580
## iter  10 value 2.056572
## iter  11 value 2.056571
## iter  12 value 2.056571
## iter  12 value 2.056571
## iter  12 value 2.056571
## final  value 2.056571
## converged
## initial  value 2.051683
## iter   2 value 2.042636
## iter   3 value 2.041905
## iter   4 value 2.041869
## iter   5 value 2.041828
## iter   6 value 2.041828
## iter   7 value 2.041828
## iter   7 value 2.041828
## iter   7 value 2.041828
## final  value 2.041828
## converged
## <><><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ar1   -0.2946 0.0793  -3.7141   3e-04
## ma1   -0.6691 0.0592 -11.3054   0e+00
## sma1  -0.9100 0.0843 -10.7971   0e+00
##
## sigma^2 estimated as 53.62596 on 212 degrees of freedom
##
## AIC = 6.958742  AICc = 6.959271  BIC = 7.021452
##
```




```
fit7 <- sarima(wheatflourTS.train, p=1,d=1,q=0,P=1,D=1,Q=1,S=12)
```

```
## initial  value 2.524928
## iter   2 value 2.268161
## iter   3 value 2.260139
## iter   4 value 2.254538
## iter   5 value 2.239487
## iter   6 value 2.232714
## iter   7 value 2.229143
## iter   8 value 2.227071
## iter   9 value 2.216486
## iter  10 value 2.212027
## iter  11 value 2.211953
## iter  12 value 2.211638
## iter  13 value 2.211638
## iter  13 value 2.211638
## iter  13 value 2.211638
## final  value 2.211638
## converged
## initial  value 2.177152
## iter   2 value 2.154789
## iter   3 value 2.151168
## iter   4 value 2.149059
## iter   5 value 2.148959
## iter   6 value 2.148953
```

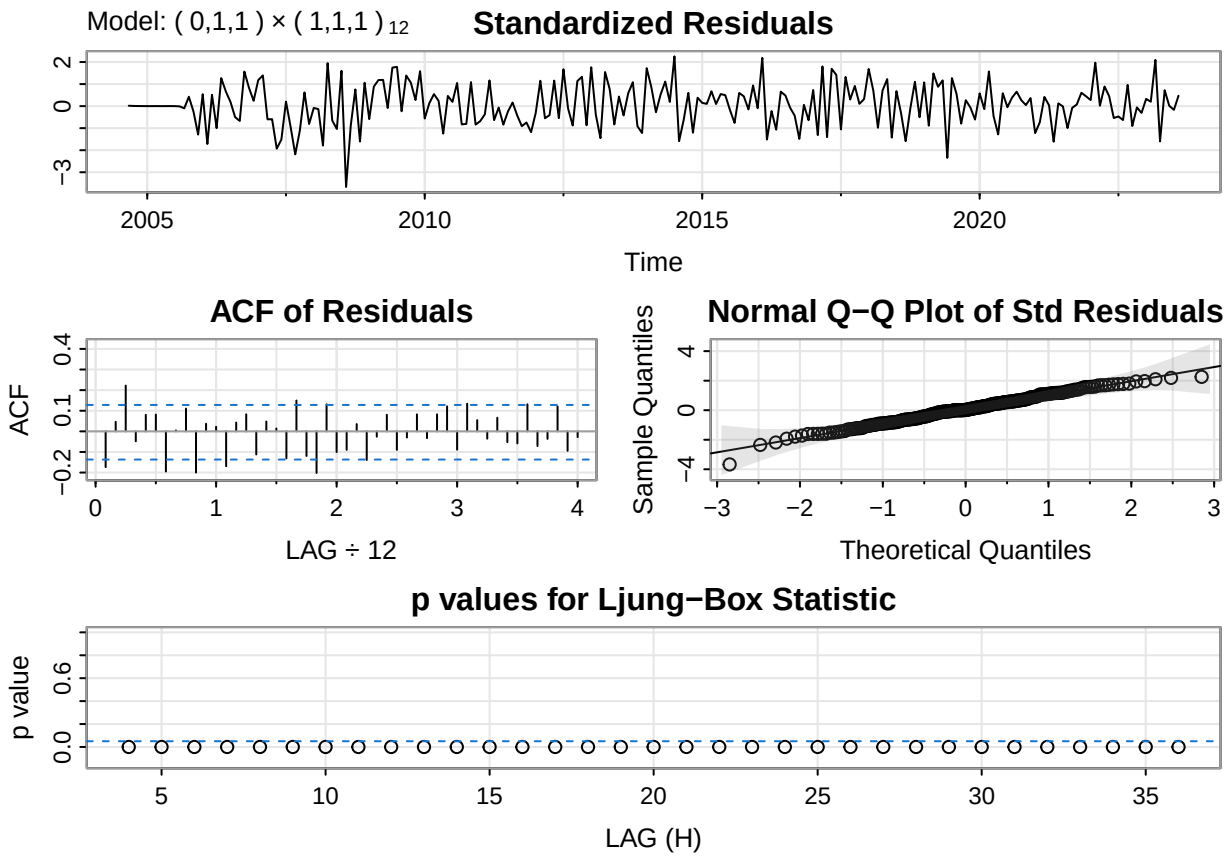
```
## iter   7 value 2.148953
## iter   7 value 2.148953
## iter   7 value 2.148953
## final  value 2.148953
## converged
## <><><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ar1   -0.5879 0.0552 -10.6458   0e+00
## sar1   0.2713 0.0727   3.7338   2e-04
## sma1  -1.0000 0.0630 -15.8673   0e+00
##
## sigma^2 estimated as 64.12067 on 212 degrees of freedom
##
## AIC = 7.172992  AICc = 7.173521  BIC = 7.235702
##
```




```
fit8 <- sarima(wheatflourTS.train, p=0,d=1,q=1,P=1,D=1,Q=1,S=12)
```

```
## initial  value 2.526504
## iter   2 value 2.209108
## iter   3 value 2.177996
## iter   4 value 2.165808
## iter   5 value 2.150904
```

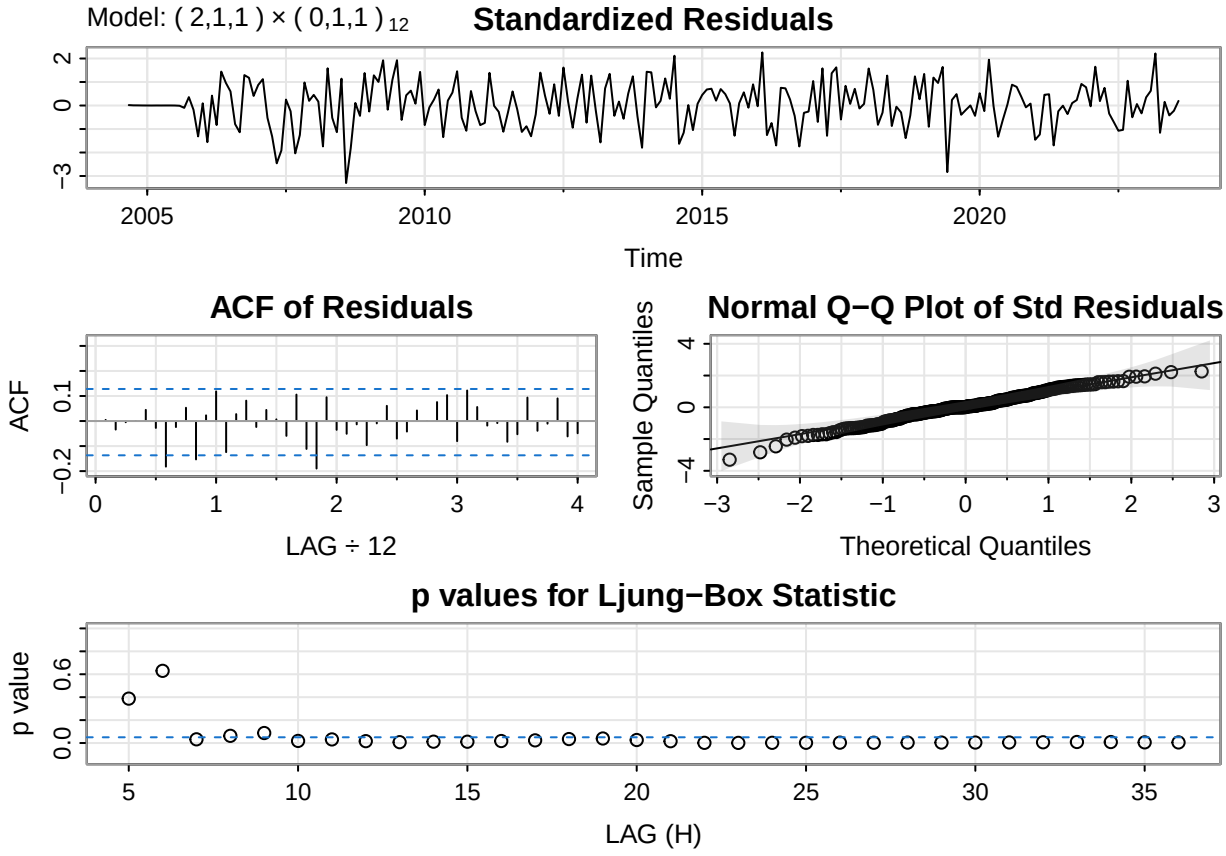
```
## iter   6 value 2.134885
## iter   7 value 2.127885
## iter   8 value 2.127258
## iter   9 value 2.127026
## iter  10 value 2.126948
## iter  11 value 2.126940
## iter  12 value 2.126932
## iter  12 value 2.126932
## iter  12 value 2.126932
## final  value 2.126932
## converged
## initial  value 2.082828
## iter   2 value 2.059794
## iter   3 value 2.053541
## iter   4 value 2.051393
## iter   5 value 2.051172
## iter   6 value 2.050842
## iter   7 value 2.050834
## iter   8 value 2.050833
## iter   9 value 2.050833
## iter  10 value 2.050833
## iter  10 value 2.050833
## iter  10 value 2.050833
## final  value 2.050833
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ma1   -0.7776 0.0403 -19.2959  0.0000
## sar1   0.2128 0.0725   2.9343  0.0037
## sma1  -1.0000 0.0930 -10.7549  0.0000
##
## sigma^2 estimated as 52.22107 on 212 degrees of freedom
##
## AIC = 6.976752  AICc = 6.977281  BIC = 7.039461
##
```



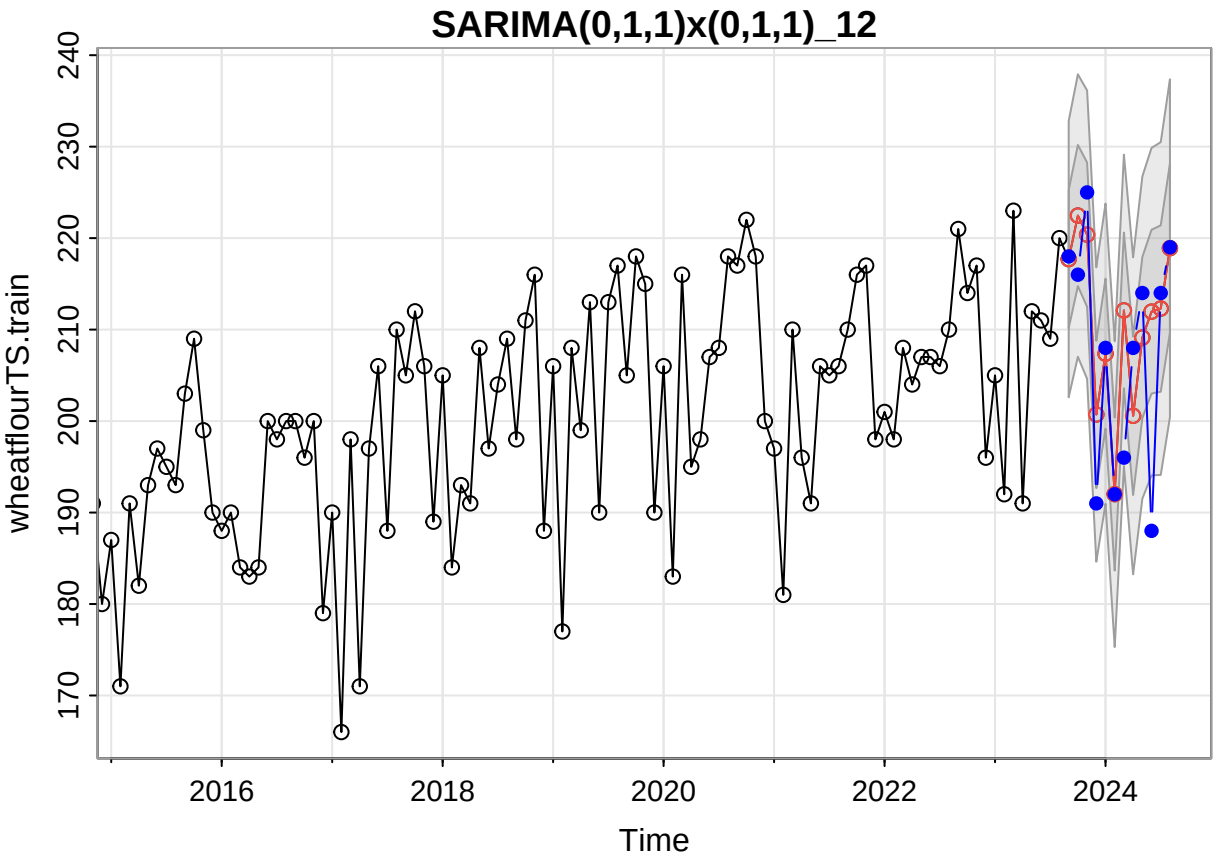

```
fit9 <- sarima(wheatflourTS.train, p=2,d=1,q=1,P=0,D=1,Q=1,S=12)
```

```
## initial  value 2.527015
## iter   2 value 2.197806
## iter   3 value 2.086544
## iter   4 value 2.057558
## iter   5 value 2.040664
## iter   6 value 2.037536
## iter   7 value 2.034117
## iter   8 value 2.032466
## iter   9 value 2.030544
## iter  10 value 2.030453
## iter  10 value 2.030453
## final  value 2.030453
## converged
## initial  value 2.024836
## iter   2 value 2.016992
## iter   3 value 2.016823
## iter   4 value 2.016758
## iter   5 value 2.016735
## iter   6 value 2.016723
## iter   7 value 2.016712
## iter   8 value 2.016711
## iter   8 value 2.016711
## iter   8 value 2.016711
```

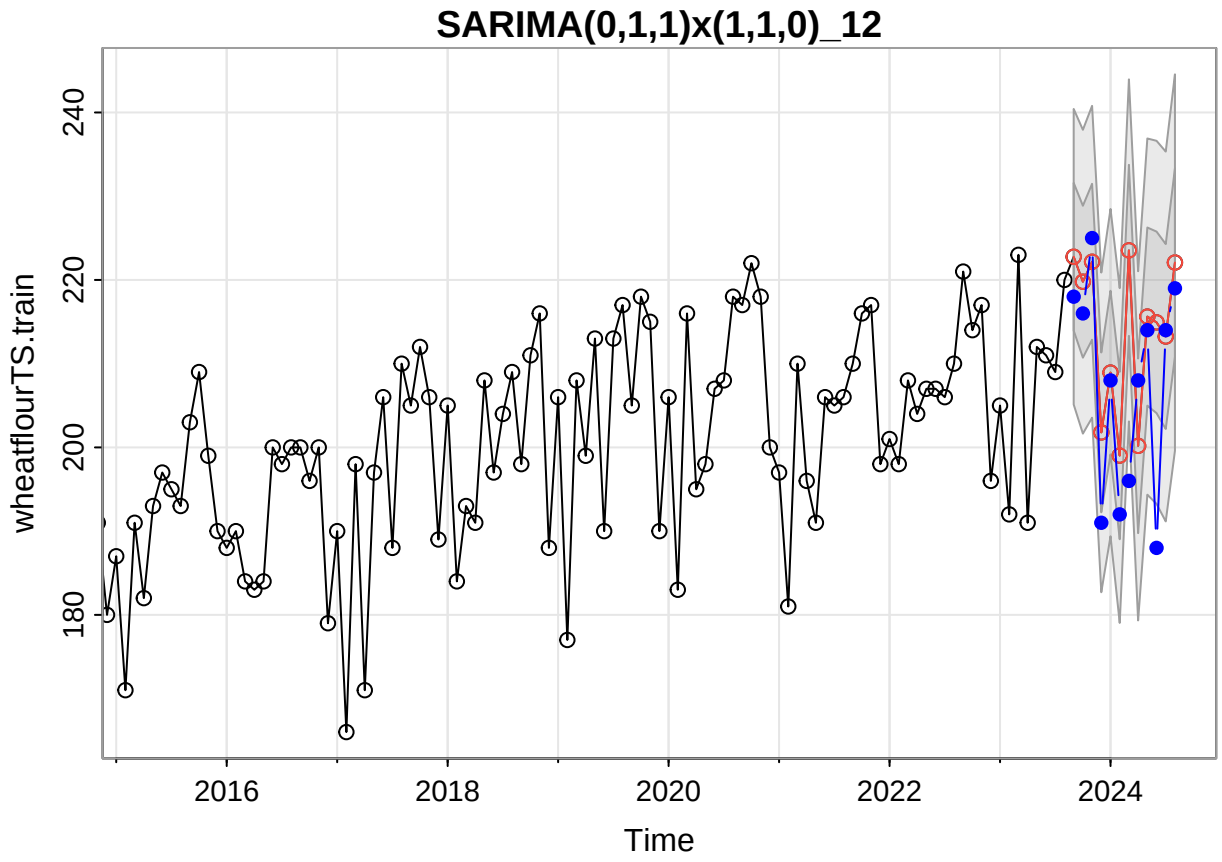
```
## final  value 2.016711
## converged
## <><><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ar1   -0.6111 0.1170  -5.2226  0.0000
## ar2   -0.3495 0.0945  -3.6967  0.0003
## ma1   -0.3757 0.1220  -3.0796  0.0023
## sma1  -0.8895 0.0729 -12.1979  0.0000
##
## sigma^2 estimated as 51.49069 on 211 degrees of freedom
##
## AIC = 6.917811  AICc = 6.918697  BIC = 6.996198
##
```



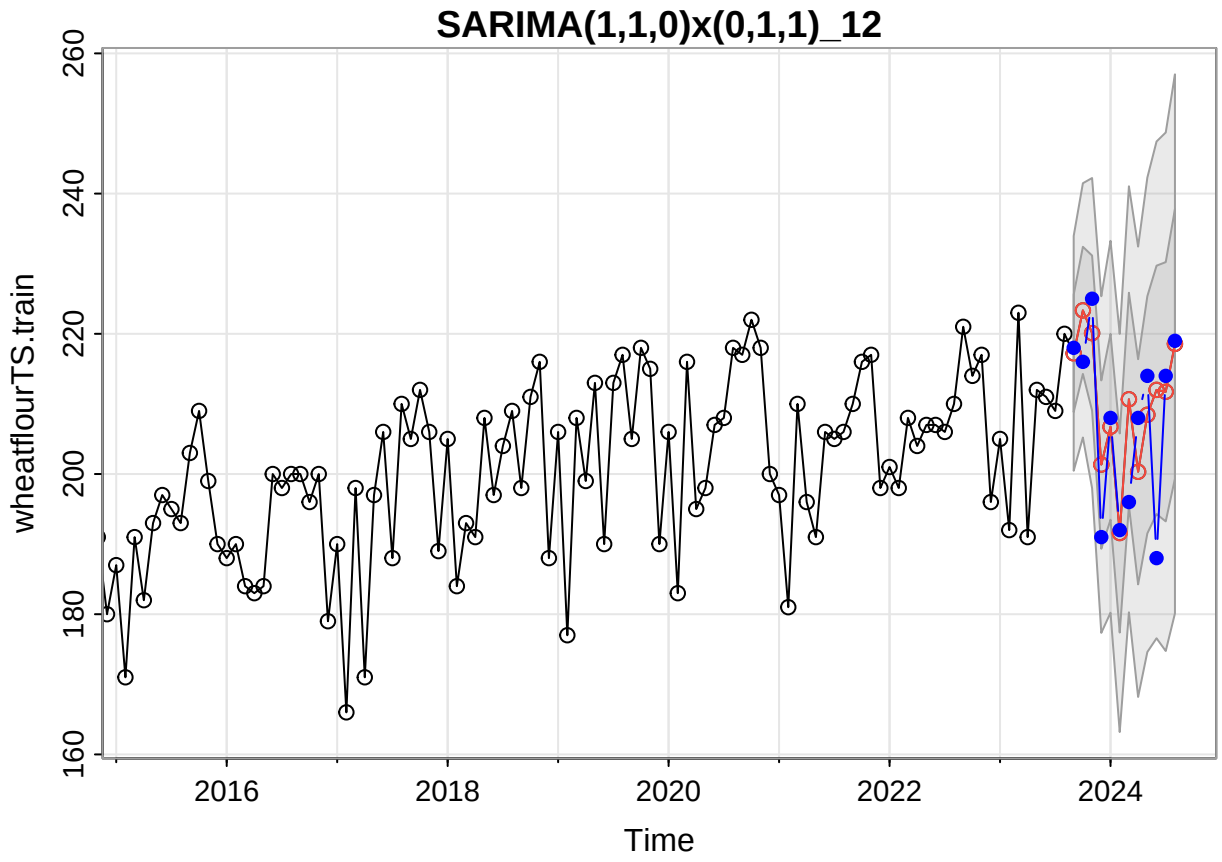

```
# Note that all models do not perform well on the pvalues.
# We will still choose the best among these according to the APSE
fore1 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=0,d=1,q=1,P=0,D=1,Q=1,S=12)
title("SARIMA(0,1,1)x(0,1,1)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```



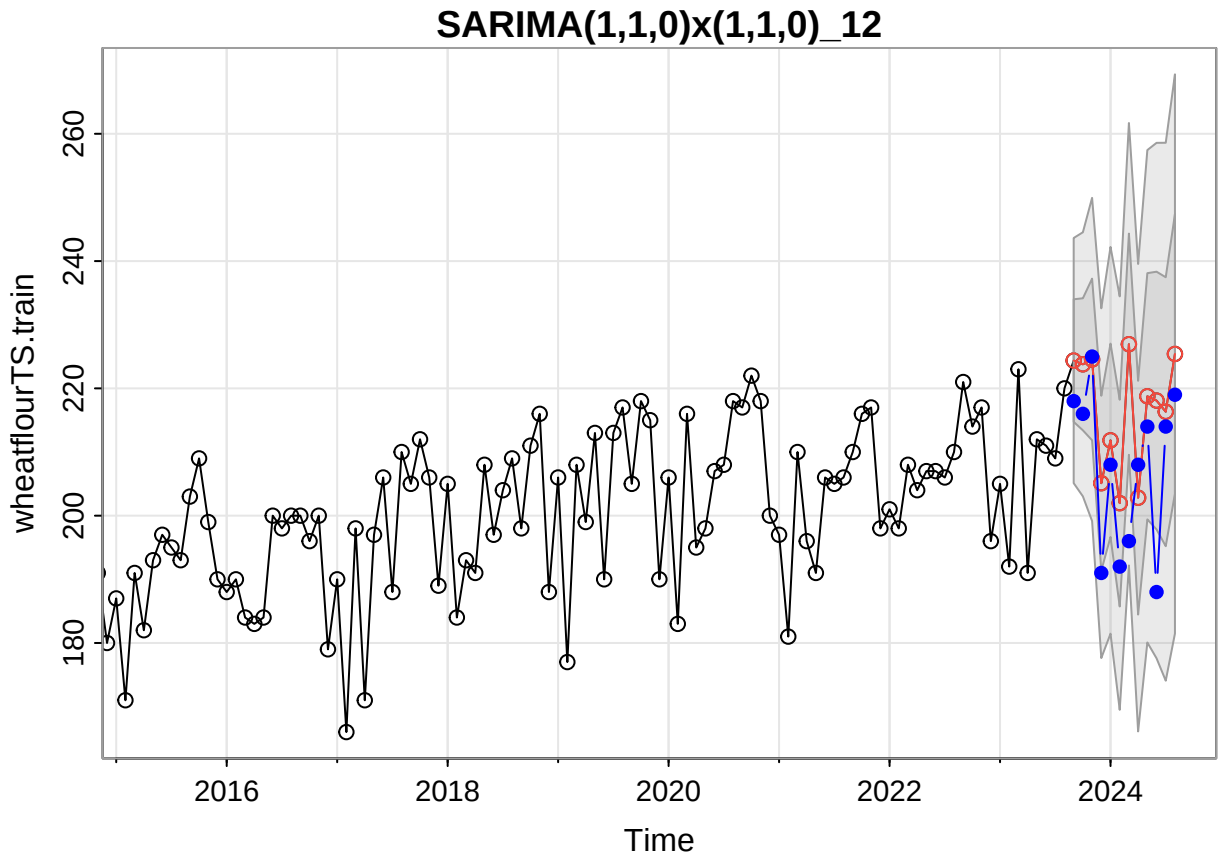

```
fore2 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=0,d=1,q=1,P=1,D=1,Q=0,S=12)
title("SARIMA(0,1,1)x(1,1,0)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```



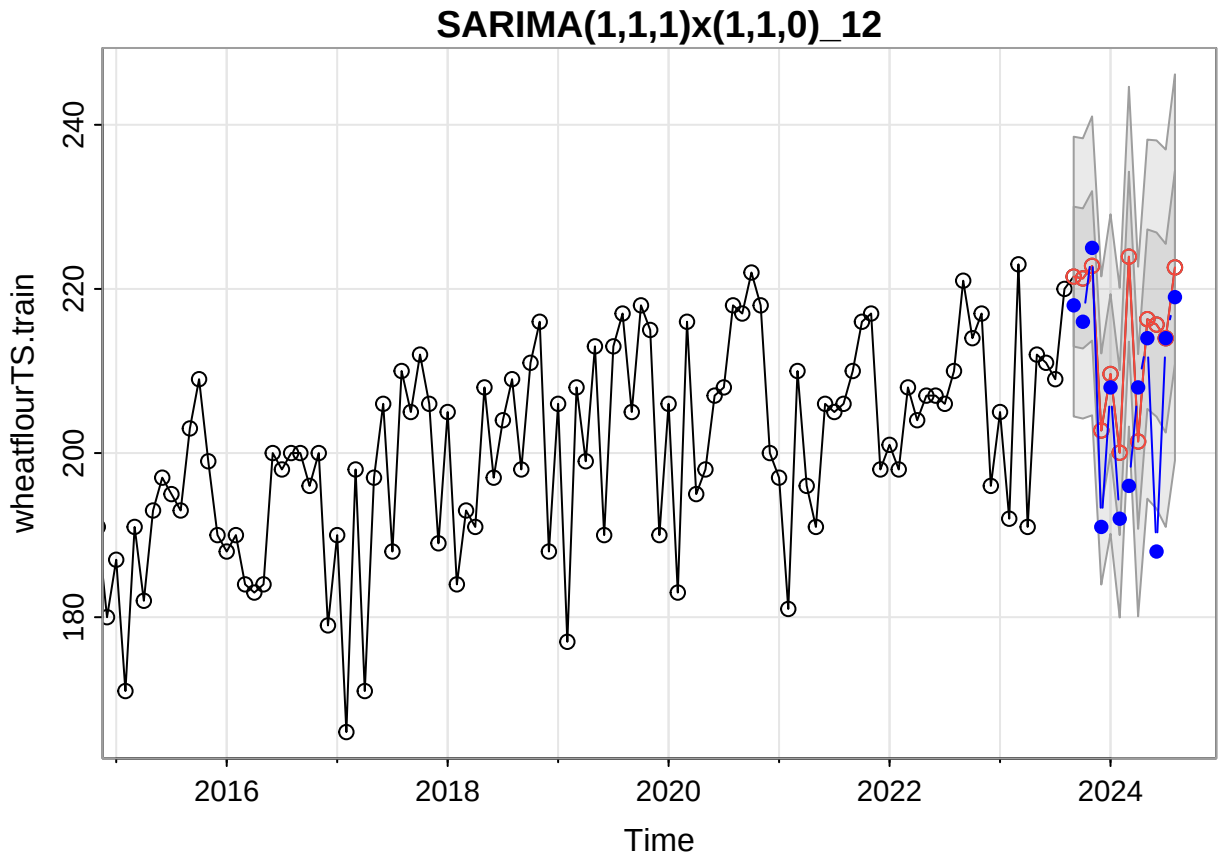

```
fore3 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=1,d=1,q=0,P=0,D=1,Q=1,S=12)
title("SARIMA(1,1,0)x(0,1,1)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```



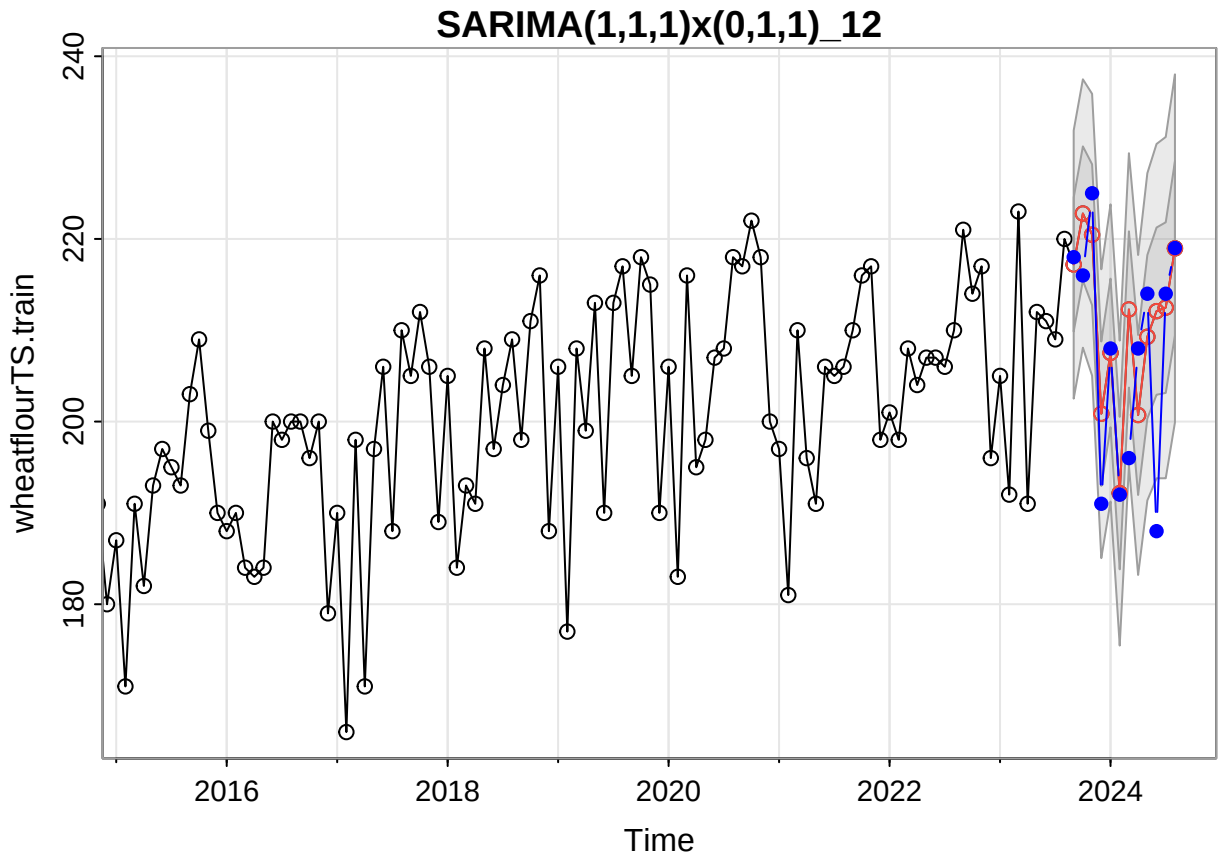

```
fore4 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=1,d=1,q=0,P=1,D=1,Q=0,S=12)
title("SARIMA(1,1,0)x(1,1,0)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```



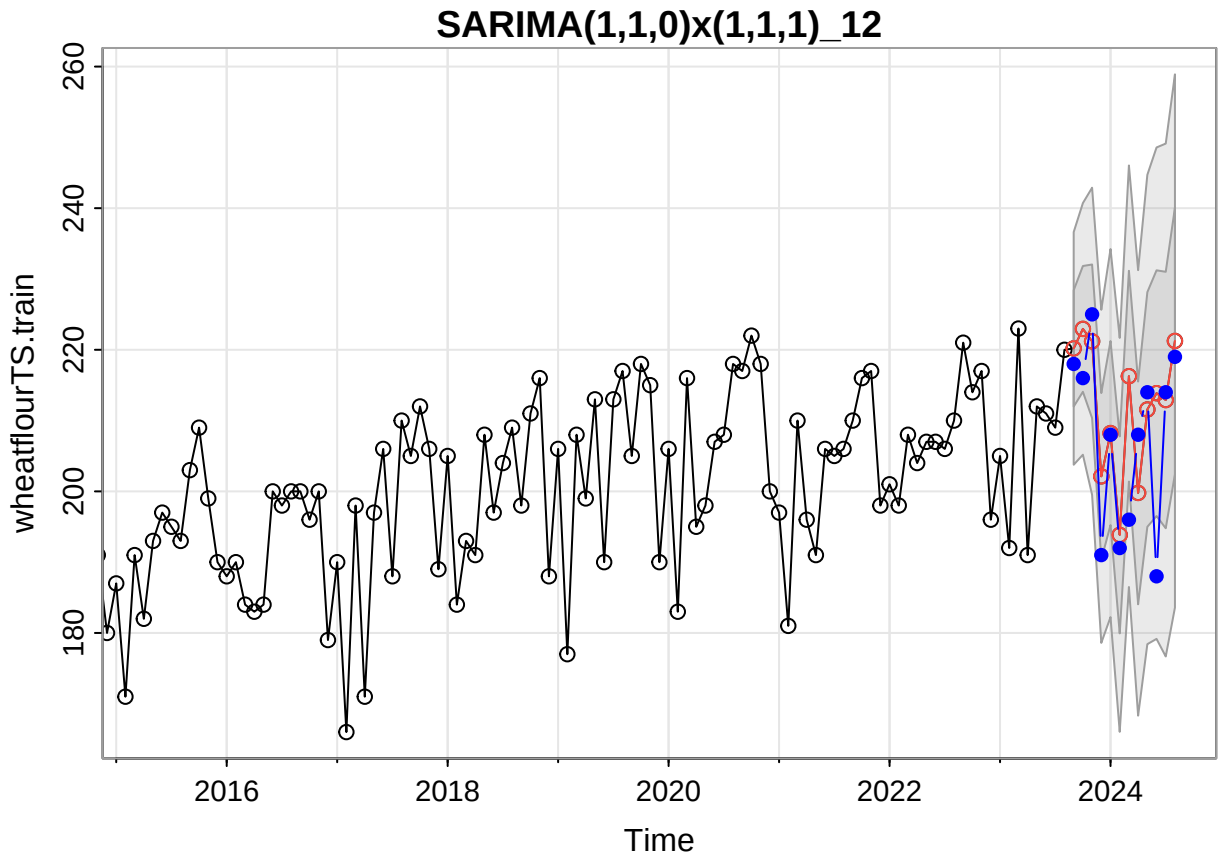

```
fore5 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=1,d=1,q=1,P=1,D=1,Q=0,S=12)
title("SARIMA(1,1,1)x(1,1,0)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```



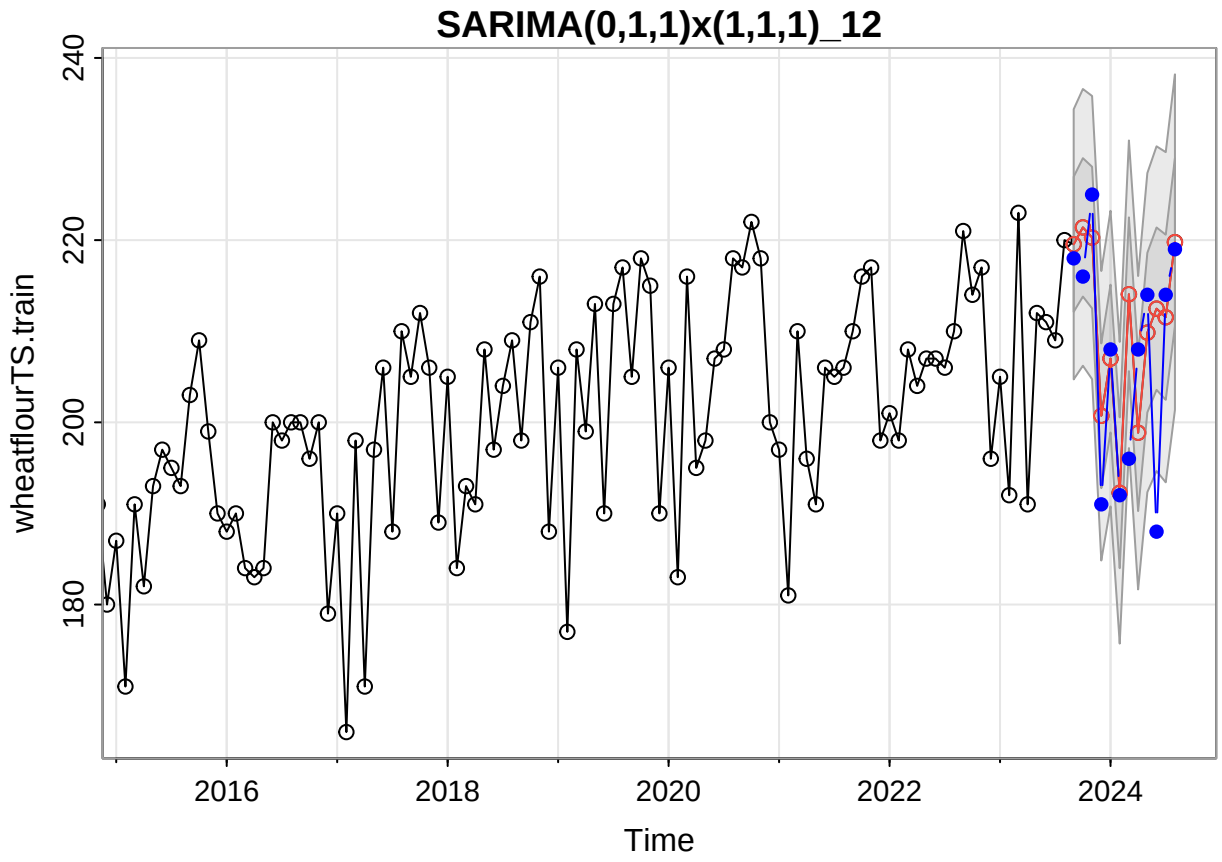

```
fore6 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=1,d=1,q=1,P=0,D=1,Q=1,S=12)
title("SARIMA(1,1,1)x(0,1,1)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```



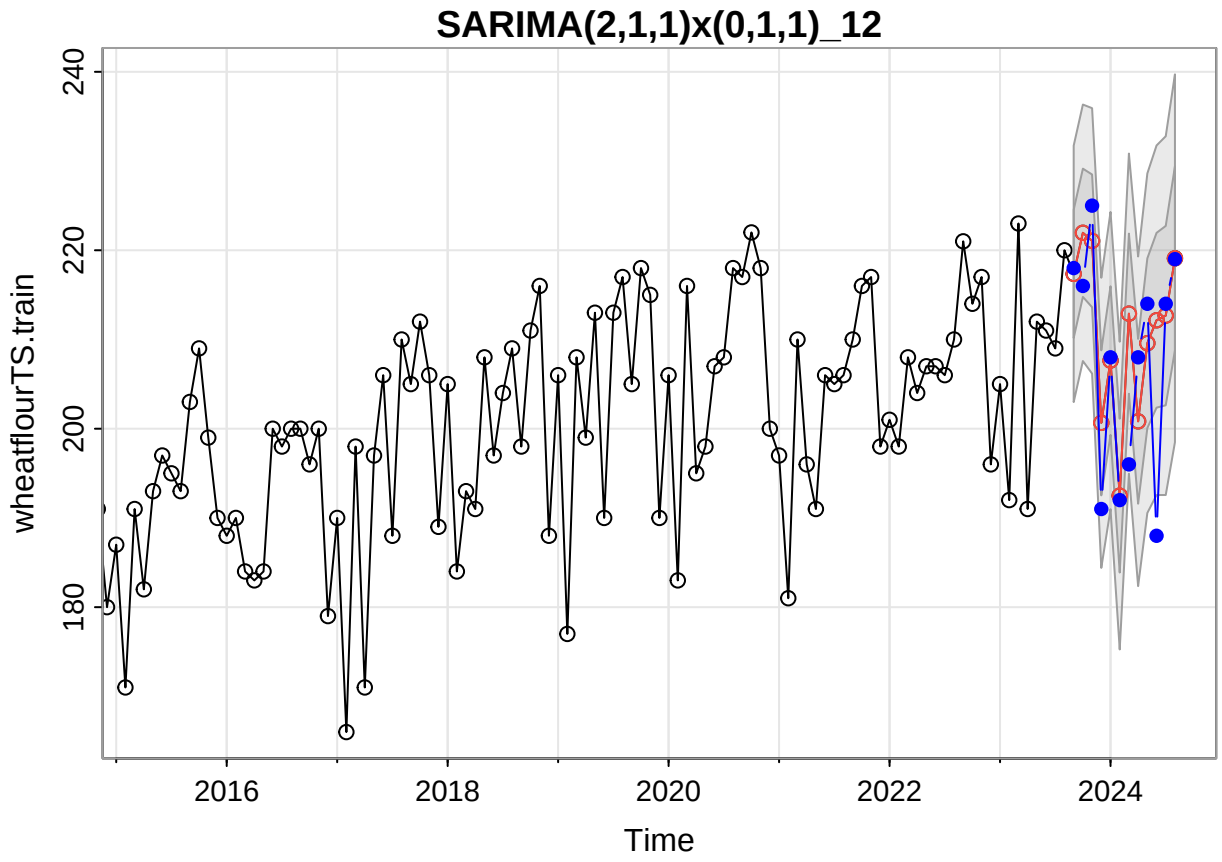

```
fore7 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=1,d=1,q=0,P=1,D=1,Q=1,S=12)
title("SARIMA(1,1,0)x(1,1,1)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```




```
fore8 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=0,d=1,q=1,P=1,D=1,Q=1,S=12)
title("SARIMA(0,1,1)x(1,1,1)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```




```
fore9 <- sarima.for(wheatflourTS.train, n.ahead=12,
                    p=2,d=1,q=1,P=0,D=1,Q=1,S=12)
title("SARIMA(2,1,1)x(0,1,1)_12")
lines(wheatflourTS.test,col='blue',type='b',pch=16)
```

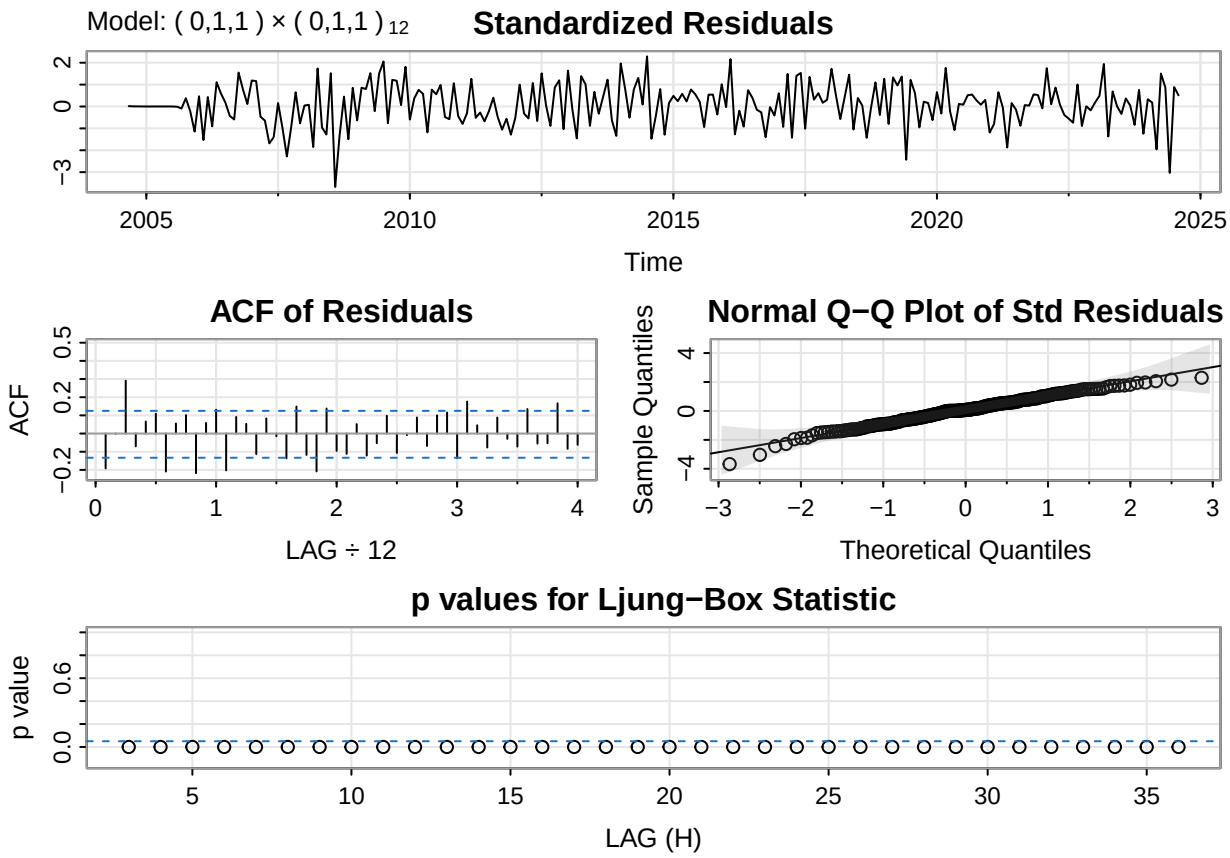



```
# Calculate APSE
APSE1 <- mean((wheatflourTS.test-fore1$pred)^2)
APSE2 <- mean((wheatflourTS.test-fore2$pred)^2)
APSE3 <- mean((wheatflourTS.test-fore3$pred)^2)
APSE4 <- mean((wheatflourTS.test-fore4$pred)^2)
APSE5 <- mean((wheatflourTS.test-fore5$pred)^2)
APSE6 <- mean((wheatflourTS.test-fore6$pred)^2)
APSE7 <- mean((wheatflourTS.test-fore7$pred)^2)
APSE8 <- mean((wheatflourTS.test-fore8$pred)^2)
APSE9 <- mean((wheatflourTS.test-fore9$pred)^2)

APSE <- c(APSE1, APSE2, APSE3, APSE4, APSE5, APSE6, APSE7, APSE8, APSE9)
data.frame(Model=1:9, APSE=APSE)
```

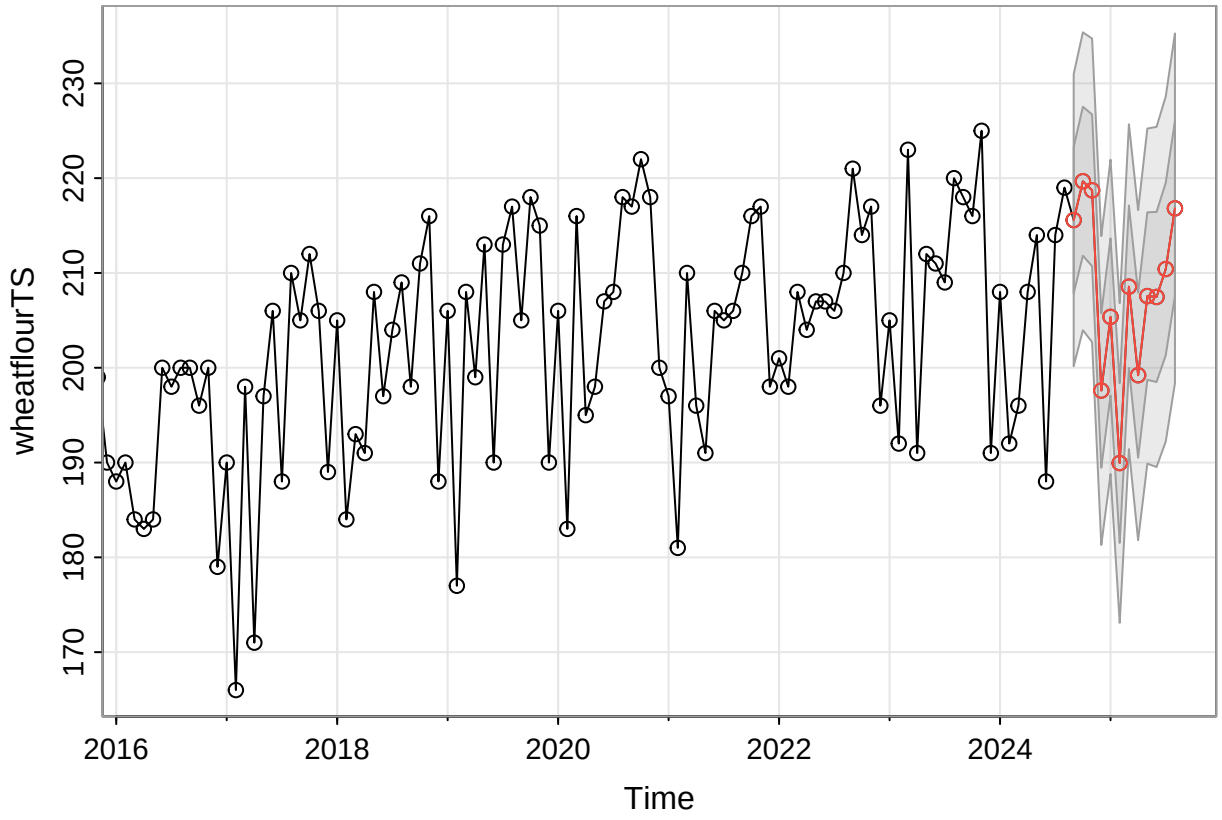
```
##   Model      APSE
## 1     1  89.45241
## 2     2 147.41085
## 3     3  89.45845
## 4     4 197.72901
## 5     5 154.75973
## 6     6  90.64788
## 7     7 112.88104
## 8     8  98.52557
## 9     9  90.55253
```

```r
# We see fit1 has the lowest APSE, we will choose it for prediction.
final.fit <- sarima(wheatflourTS, p=0,d=1,q=1,P=0,D=1,Q=1,S=12)
```

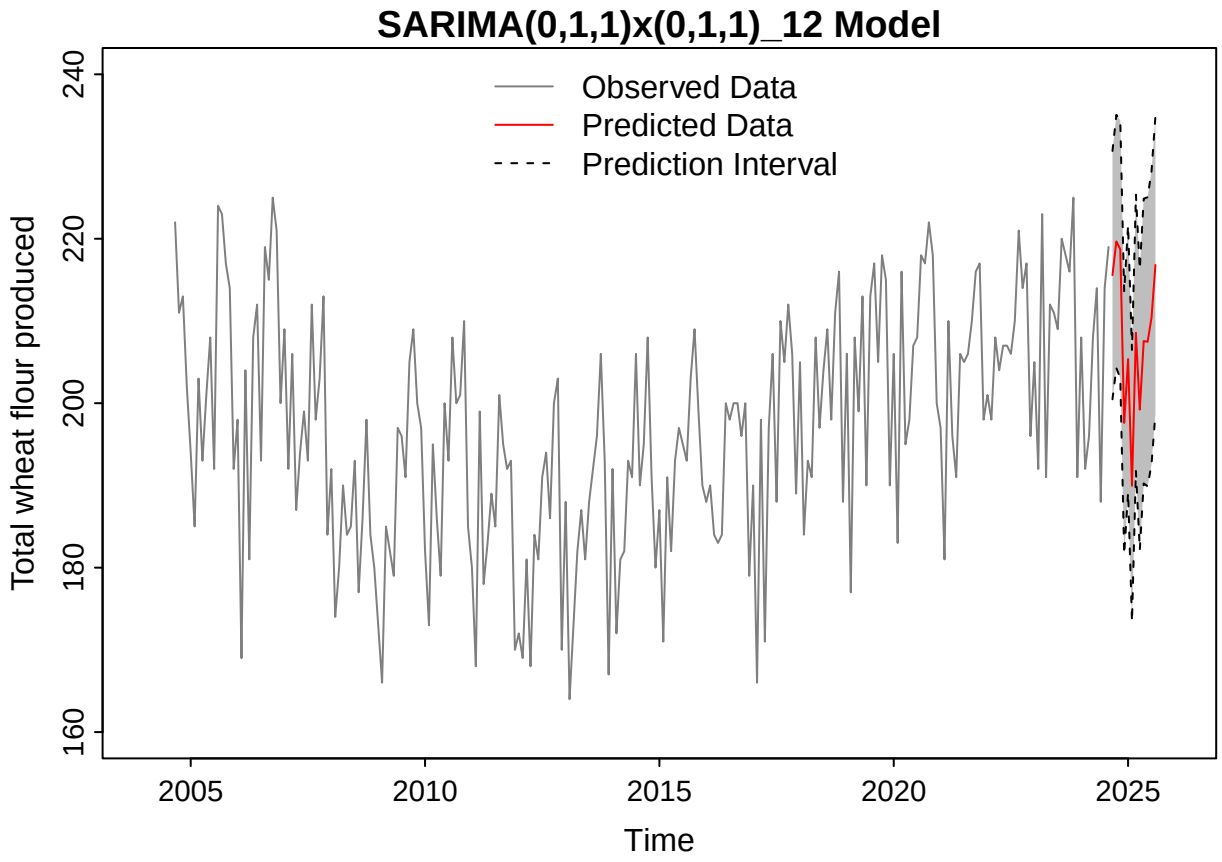
```
## initial  value 2.563134
## iter   2 value 2.224122
## iter   3 value 2.166864
## iter   4 value 2.143147
## iter   5 value 2.104441
## iter   6 value 2.103533
## iter   7 value 2.100342
## iter   8 value 2.099488
## iter   9 value 2.099384
## iter  10 value 2.099361
## iter  11 value 2.099354
## iter  12 value 2.099354
## iter  12 value 2.099354
## iter  12 value 2.099354
## final  value 2.099354
## converged
## initial  value 2.096396
## iter   2 value 2.087544
## iter   3 value 2.087251
## iter   4 value 2.087198
## iter   5 value 2.087198
## iter   5 value 2.087198
## iter   5 value 2.087198
## final  value 2.087198
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##      Estimate     SE  t.value p.value
## ma1   -0.8013 0.0360 -22.2732       0
## sma1  -0.9049 0.0685 -13.2075       0
##
## sigma^2 estimated as 59.14481 on 225 degrees of freedom
##
## AIC = 7.038705  AICc = 7.038941  BIC = 7.083968
##
```



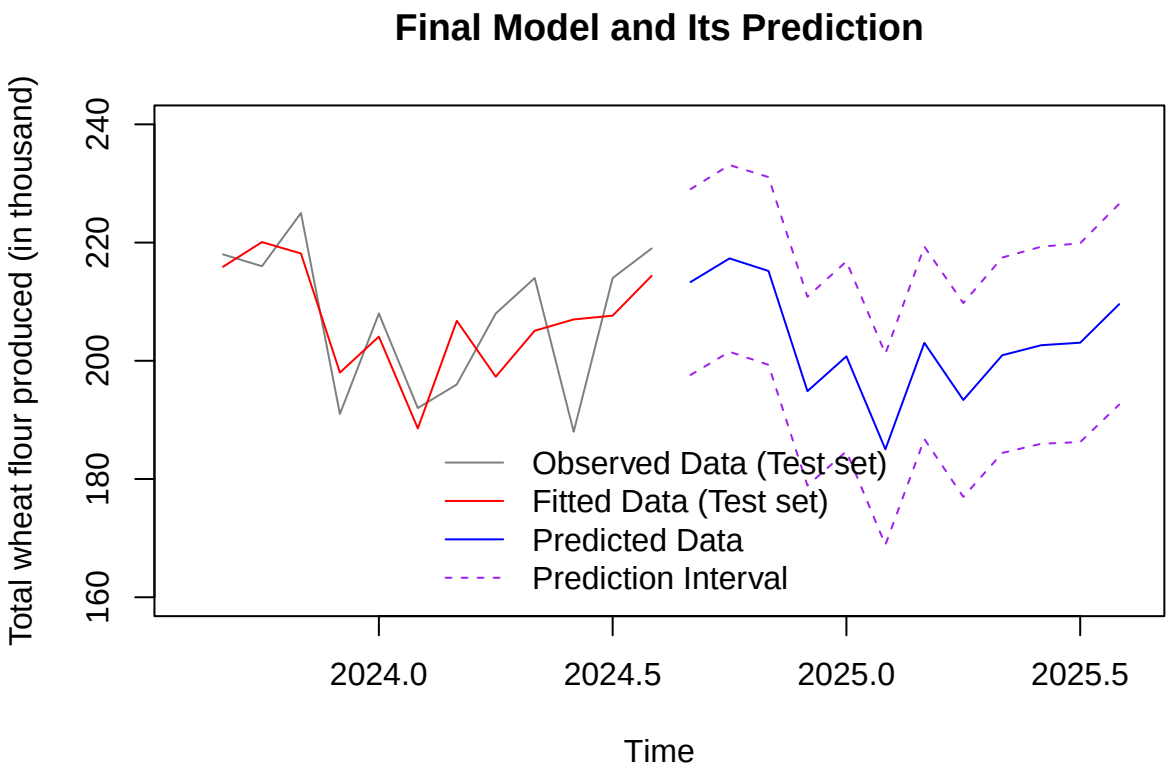

```
future.forecast <- sarima.for(wheatflourTS, n.ahead=12, p=0,d=1,q=1,P=0,D=1,Q=1,S=12)
```




```
fit <- future.forecast$pred # predicted data
lower <- fit-1.96*future.forecast$se # 95% prediction interval
upper <- fit+1.96*future.forecast$se # 95% prediction interval
ts.plot(wheatflourTS, xlim=c(2004, 2026), ylim=c(160, 240), col=adjustcolor("black", 0.5),
        ylab="Total wheat flour produced",main="SARIMA(0,1,1)x(0,1,1)_12 Model")

x = c(time(upper) , rev(time(upper)))
y = c(upper, rev(lower))
polygon(x, y, col="grey" , border=NA) # shade prediction interval

lines(fit,col='red',type='l',pch=16 , cex=0.5)
lines(lower,col='black',lty=2)
lines(upper,col='black',lty=2)
legend("top", legend = c("Observed Data", "Predicted Data", "Prediction Interval"),
       col=c(adjustcolor("black", 0.5), "red", "black"),
       lty=c(1, 1, 2), bty="n")
```




```r
# plot our final chosen model with a clearer version
time = Time[-train.indx]
plot(Time[-as.vector(train.indx)], wheatflourTS[-as.vector(train.indx)],
     xlim = c(2023.6, 2025.6), ylim = c(160, 240),
     col = c(adjustcolor("black", 0.5)), type = "l",
     xlab = "Time", ylab = "Total wheat flour produced (in thousand)",
     main = "Final Model and Its Prediction") # observed data (test set)
lines(time, model.regression$fitted.values[-train.indx], col = "red") # fitted data (test set)
lines(Fitted.predict.regression[, 1] ~ predict$Time, col = "blue") # predicted data
lines(Fitted.predict.regression[, 2] ~ predict$Time, lty = 2, col = "purple") # 95% prediction interval
lines(Fitted.predict.regression[, 3] ~ predict$Time, lty = 2, col = "purple") # 95% prediction interval
legend("bottom", legend = c("Observed Data (Test set)", "Fitted Data (Test set)",
                            "Predicted Data", "Prediction Interval"),
       col = c(adjustcolor("black", 0.5), "red", "blue", "purple"), lty = c(1, 1, 1, 2), bty = "n")
```


Final Model and Its Prediction
Total wheat flour produced (in thousand)
160
180
200
220
240
2024.0
2024.5
2025.0
2025.5
Time
Observed Data (Test set)
Fitted Data (Test set)
Predicted Data
Prediction Interval